

Titre Professionnel DWWM

DOSSIER DE PROJET

Développeur Web & Web Mobile

APPLICATION DÉVELOPPÉE & PRÉSENTÉE

CICADOS

*Plateforme Web de Réservation de Tables & Gestion d'Événements pour Bar à Jeux et
Duels TCG*

Candidat :

PARRAUD OLIVIER

Développeur Fullstack Web & Mobile

Établissement & Référentiel :

La Plateforme — DWWM

Référentiel REAC — Ministère du Travail

SOMMAIRE

1. Introduction	4
1.1 Parcours et reconversion	4
1.2 Formation La Plateforme	4
2. Cahier des Charges du Projet	5
2.1 Genèse et évolution de Cicados	5
2.2 Problématiques logistiques & solutions	5
2.3 Typologie des Personas	7
2.4 Matrice de priorisation MoSCoW	8
2.5 User Stories de l'application	9
3. Conception Technique & Fonctionnelle	20
3.1 Arborescence du site (Sitemap)	20
3.2 Dictionnaire des routes (Front & Back)	21
3.3 Modélisation BDD (MCD, MLD, MPD)	29
3.4 Dictionnaire de données	42
4. Spécifications Techniques & Stack	45
4.1 Choix et justification de la stack	45
4.2 Infrastructure, Conteneurisation	48
4.3 Gestion Agile (Scrum et Sprints)	50
4.4 Conventions Git, commits et versioning	50
5. Réalisations & Étude du Code Source	51
5.1 Frontend : Profile.jsx (Relance support)	51
5.2 Frontend : ContactModal.jsx (Events)	52
5.3 Backend : message.model.js (SQL)	53
5.4 Backend : auth & admin.middleware (JWT/RBAC)	54
5.5 Backend : boardgame.controller (Failover JSON)	55
5.6 Backend : bgg.controller (Proxy XML/JSON)	56
6. Focus Algorithmique & Avancé	57
6.1 Algorithme anti-surbooking réservation	57
6.2 Proxy BGG et conversion XML vers JSON	60
6.3 Basculement de secours JSON local	60
6.4 Messagerie support & Sync Temps Réel	60
6.5 État React et isolation des modales	60
7. Jeux d'Essai et Scénarios de Validation	61
7.1 Scénario A : Inscription et Conflit de Table	61
7.2 Scénario B : Jauges & Inscription Tournoi	61
7.3 Scénario C : Discussion Support Multi-Tours	61
8. Veille Technologique & Sécurité (OWASP)	62
9. Ressources Anglophones	62
10. Conclusion et Perspectives	63

10.1 Conclusion Générale & Bilan	63
10.2 Synthèse du Projet & Compétences	63
10.3 Perspectives d'Évolution (V2)	63
11. Annexes du Dossier	63
11.1 Annexe 1 : Script SQL schema.sql	63
11.2 Annexe 2 : Figma Wireframes & UI	67
11.3 Annexe 3 : Tests E2E Puppeteer	70
11.4 Annexe 4 : Variables env (.env.example)	71

1 — INTRODUCTION

1.1 Parcours professionnel et reconversion

Mon parcours est marqué par une expérience significative dans l'industrie de l'informatique. J'ai étudié l'informatique en obtenant un diplôme professionnel (**BAC Système Numérique - Option réseaux**). Cette aventure m'a permis d'acquérir de solides compétences de gestion administrative, de maîtrise de logiciel, de compréhension de l'univers du développement et de résolution rapide de problèmes.

La crise sanitaire liée à la pandémie de **COVID-19** a profondément affecté la suite de mon parcours scolaire. Durant les périodes de fermeture administrative et de confinement, j'ai été amené à réfléchir à mon avenir professionnel. J'ai alors cherché une reconversion qui me permettrait de valoriser mes compétences de gestion et d'organisation tout en m'ouvrant les portes d'un domaine porteur, créatif et tourné vers l'avenir. C'est durant cette période de transition que j'ai commencé à m'intéresser de près au développement informatique. J'ai compris que le développement web m'offrait l'opportunité de lier la rigueur logique de la conception *backend* à l'aspect visuel et interactif du *frontend*.

1.2 Le choix du développement web et de la formation La Plateforme

Afin de concrétiser cette reconversion et d'acquérir un niveau technique professionnel, j'ai choisi de suivre la formation de l'école **La Plateforme**. L'approche pédagogique en présentiel m'a permis de me consacrer pleinement à l'apprentissage du développement dans un cadre structuré et exigeant, reproduisant fidèlement les conditions du travail en équipe de développement moderne (méthodologie **Scrum**, outils de versioning collaboratif **Git/GitHub**, réunions quotidiennes).

La formation s'est découpée en trois phases majeures :

La formation socle : Apprentissage approfondi des bases du web (*HTML5, CSS3, JavaScript moderne ES6+*), de la modélisation et de la structuration de bases de données **SQL**, ainsi que de l'architecture **MVC**.

La spécialisation : Focus complet sur un framework moderne (**React 18**), me permettant d'appréhender la gestion réactive de l'état global et la création de composants réutilisables complexes.

Le projet professionnel de fin d'études : C'est dans ce cadre, équivalent au projet de fin de formation, que j'ai développé l'application **Cicados**. Ce projet personnel m'a permis de mettre en application l'ensemble des compétences théoriques et pratiques acquises, de la rédaction du cahier des charges initial jusqu'au déploiement et à la validation des scénarios de test.

2 — CAHIER DES CHARGES DU PROJET

2.1 Genèse et évolution du projet Cicados

Le concept de **Cicados** est né d'une passion personnelle pour les univers ludiques, en particulier les jeux de cartes à collectionner (TCG) comme **Magic: The Gathering**, **Pokémon**, **Yu-Gi-Oh!**, **Lorcana**, **Riftbound TCG**, ainsi que les jeux de société modernes. Les établissements hybrides, mêlant boutique, café et espace de jeu, connaissent un essor remarquable. Cependant, ces structures se heurtent fréquemment à des difficultés d'organisation logistique et de communication client.

Initialement pensé comme un outil de réservation de tables et d'inscriptions aux tournois, **Cicados** s'est enrichi de fonctionnalités majeures pour couvrir l'ensemble du besoin opérationnel d'un café-boutique moderne :

- Un système de messagerie et support client bidirectionnel permettant aux membres d'interagir directement avec les administrateurs et d'entretenir un fil de discussion multi-tours.
- La gestion de boutique et du stock en temps réel pour la vente des jeux de société.
- Des modules avancés d'administration avec prévisualisation en temps réel et importation des jeux les plus populaires depuis BoardGameGeek (BGG).

2.2 Besoins, contraintes, problématiques logistiques et solutions

1. Prévention du Surbooking

Problématique : Dans un établissement ayant une capacité d'accueil physique restreinte (4 salles de jeu dans le modèle initial), la réservation sans vérification immédiate des ressources mène inévitablement à des conflits d'horaires et à une insatisfaction des clients.

Besoin : Garantir qu'aucune table ne soit réservée deux fois sur un même créneau horaire.

Solution : J'ai conçu un algorithme backend d'intersection de créneaux temporels (`start_time < ? AND end_time > ?`) qui vérifie l'occupation en temps réel avant de m'attribuer dynamiquement la première table disponible.

2. Communication et Support Client Intégré

Problématique : Les joueurs et clients ont fréquemment des questions spécifiques (règles de tournoi, possibilité d'apporter son propre tapis de jeu, horaires spéciaux, disponibilité en boutique). Sans canal dédié, ils utilisent des e-mails externes ou des réseaux sociaux dispersés.

Besoin : Offrir aux utilisateurs connectés un moyen simple d'envoyer un message aux administrateurs depuis n'importe quelle page du site, de consulter l'historique des échanges depuis leur profil et de pouvoir répondre/relancer la conversation. Côté administrateur, disposer d'un onglet centralisé pour lire, répondre et suivre l'état de chaque demande avec notifications visuelles.

Solution : J'ai créé une fenêtre modale de contact déclenchable via un bouton « Contacter » (visible au-dessus de la FAQ). J'ai ensuite conçu la table SQL `messages` avec un suivi d'état (`is_read`, `admin_reply`, `replied_at`, `user_read`). Puis J'ai implémenté un onglet « Messages » dans le Dashboard Admin avec des filtres, une recherche et un formulaire de réponse rapide.

J'ai ajouté une section « Mes Messages & Support » sur la page Profil qui affiche mes questions et les réponses en texte blanc, et j'y ai inclus un bouton « Répondre » ouvrant un formulaire de relance (discussion multi-tours).

J'ai mis en place un système de notifications réactives avec des badges dynamiques (compteur rouge sur l'avatar du Header et sur l'onglet admin), alimenté par l'événement personnalisé `messages_updated`.

3. Gestion de la Boutique et des Stocks de Jeux

Problématique : En plus de la location de tables et des événements, l'établissement vend des jeux de société et des produits des TCG partenaires avec la boutique. Si un jeu est en rupture de stock, les clients risquent d'être déçus en boutique.

Besoin : Afficher la disponibilité des jeux sur le catalogue public et permettre au gérant d'ajuster le stock en un clic depuis l'administration.

Solution : J'ai ajouté la colonne `stock` dans la table `board_games`. J'ai mis en place l'affichage d'un badge dynamique sur les cartes du catalogue : « En stock : X » ou « Rupture de stock ». J'ai développé un système de contrôle rapide du stock sur le Dashboard Admin grâce à des boutons d'incrément et de décrémentation instantanés (+ / -) fonctionnant via des requêtes AJAX.

4. Notification Pastille Admin & Accessibilité Typographique

Problématique : L'administrateur doit se rendre dans son onglet des messages pour vérifier les nouvelles demandes. Pour une meilleure lisibilité des horaires, afficher clairement les

créneaux d'ouverture de la boutique. Harmoniser toutes les pages du site en un seul et même thème.

Besoin : Signaler instantanément les messages non lus dans la barre de navigation et améliorer la lisibilité globale. Afficher les horaires sur la page d'accueil des tournois ou des événements proposés par le site.

Solution : J'ai intégré une pastille rouge fixe sur le bouton Admin de la Navbar (desktop et mobile), synchronisée via l'événement `messages_updated`. J'ai mis en valeur les dates et heures sur les cartes de tournois et d'événements en jaune ambré. J'ai harmonisé les en-têtes de toutes les pages avec des encarts délimités aux coins arrondis, une bordure dorée et des halos lumineux ambiants en verre dépoli.

5. Importation de Ludothèque de Masse

Problématique : Saisir manuellement les caractéristiques techniques (joueurs, durée, catégorie, visuels) de 100 jeux populaires représente des dizaines d'heures de travail de saisie pour le gérant.

Besoin : Alimenter le catalogue à partir de données de référence certifiées.

Solution : J'ai importé la clé API BoardGameGeek (BGG) dans le fichier `.env`. J'interroge ensuite l'API XML2 de BoardGameGeek, puis je convertis les flux XML en JSON via `fast-xml-parser` pour insérer automatiquement les jeux les plus populaires en base de données.

2.3 Typologie de la clientèle (Personas)

Persona 1 : Sarah, la joueuse de TCG compétitif

Profil : 27 ans, informaticienne. Joueuse aguerrie de **Magic: The Gathering**.

Besoins : S'inscrire rapidement aux tournois officiels, suivre les jauges de places en temps réel, et pouvoir envoyer un message rapide à l'équipe pour préciser la liste de ses cartes (decklist) ou poser des questions sur le format.

Frustrations : Les inscriptions floues sans confirmation et l'impossibilité d'échanger avec l'organisateur.

Persona 2 : Thomas, le joueur occasionnelle de jeux de société

Profil : 34 ans, responsable marketing, père de deux enfants.

Besoins : Consulter la disponibilité des jeux en stock dans la boutique, réserver une table

adaptée le week-end, et suivre ses demandes de renseignements directement dans son espace membre.

Frustrations : Arriver sur place et trouver un jeu en rupture ou toutes les tables occupées.

Persona 3 : Olivier, le gérant de la boutique

Profil : 45 ans, passionné d'entrepreneuriat et de jeux de plateau.

Besoins : Visualiser l'ensemble des réservations et des messages clients en un coup d'œil, répondre aux questions des joueurs depuis une interface centralisée, ajuster le stock de la boutique et importer les jeux tendance BGG.

Frustrations : Devoir gérer les messages clients sur plusieurs canaux (mails, téléphone, réseaux) et saisir les fiches jeux à la main.

2.4 Matrice de priorisation MoSCoW

Le périmètre fonctionnel est défini par les User Stories présentées ci-dessous, structurées selon la méthode Agile standard et priorisées via la matrice MoSCoW (**MUST** ● : Vital, **SHOULD** ● : Important, **COULD** ● : Confort, **WON'T** ● : Exclu).

MATRICE DE PRIORISATION MoSCoW

project Cicados

1. MUST HAVE

- Authentification JWT
- Réservation anti-surbooking
- Inscriptions Tournois TCG
- Dashboard Admin (6 onglets)
- Espace Profil membre

2. SHOULD HAVE

- Messagerie & Support client multi-tours
- Gestion stock boutique (+/-)
- Importation BGG Hot (XML/JSON)
- Secours JSON (Failover)
- Upload avatars & visuels (Multer)

3. COULD HAVE

- Paiement en ligne tournois (Stripe)
- Notifications WebSockets (Socket.io)
- Filtres de recherche avancés
- Export PDF récépissé

4. WON'T HAVE

- Application mobile native (React Native)
- Vente e-commerce livraison
- Système Ligue Elo
- Carte de fidélité

2.5 User Stories de l'application

● 1. EPIC 01 — MUST HAVE (Fonctionnalités Indispensables V1)

US-01.1 | Inscription, Connexion et Authentification JWT

Priorité : MUST ● | 5 pts

En tant que **Visiteur**

Je veux **pouvoir me créer un compte puis m'authentifier**

Afin d'**accéder aux fonctionnalités de réservation de tables, d'inscriptions aux tournois et au support.**

Critères d'acceptation :

- Le mot de passe saisi doit comporter au moins 8 caractères et au moins 1 caractère spécial, haché avec `bcrypt`.

- L'authentification réussie génère un JSON Web Token (JWT) valide stocké dans le `localStorage`.
- L'adresse e-mail doit être unique dans la base de données MySQL.

US-01.2 | Réservation de Table Anti-Surbooking

Priorité : MUST ● | 8 pts

En tant que **Joueur connecté**

Je veux **sélectionner une date, une heure de début, une durée et le nombre de joueurs pour ma session**

Afin de **réserver automatiquement une table libre sans risque de surbooking**

Critères d'acceptation :

- L'algorithme SQL vérifie l'occupation en temps réel (`start_time < nouveau_end_time AND end_time > nouveau_start_time`).
- La capacité maximale est strictement limitée aux 4 tables de l'établissement. Si toutes les tables sont occupées, la réservation est refusée avec un message explicite.
- L'utilisateur peut spécifier un jeu du catalogue ou cocher "J'apporte mon propre jeu".

US-01.3 | Inscription et Désinscription aux Tournois TCG

Priorité : MUST ● | 5 pts

En tant que **Joueur connecté**

Je veux **m'inscrire à un tournoi TCG (Magic, Pokémon, Lorcana, One Piece, Yu-Gi-Oh!, Riftbound) et me désinscrire en cas d'imprévu**

Afin de **garantir ma place pour la compétition**

Critères d'acceptation :

- Le nombre d'inscrits augmente/diminue instantanément avec mise à jour réactive des jauges de capacité.
- Une contrainte SQL unique (`UNIQUE KEY`) empêche toute double inscription d'un même membre.

US-01.4 | Console d'Administration Centralisée (CRUD)

Priorité : MUST ● | 8 pts

En tant qu'**Administrateur**

Je veux **accéder à un tableau de bord sécurisé structuré en 6 onglets (Réservations, Tournois, Événements, Jeux, Utilisateurs, Messages)**

Afin de **gérer l'ensemble des données de l'établissement**

Critères d'acceptation :

- Accès strictement restreint aux utilisateurs possédant le rôle ADMIN.

US-01.5 | Espace Profil & Historique des Activités

Priorité : MUST ● | 3 pts

En tant que **Joueur connecté**

Je veux **consulter mes données personnelles, mettre à jour mon avatar et visualiser l'historique de mes réservations et tournois**

Afin de **suivre l'ensemble de mes activités au sein du café-boutique**

Critères d'acceptation :

- Une interface lisible et claire pour l'utilisateur affichant réservations, tournois et espace de support client.

US-01.6 | Navigation Responsive & Changement de Langue FR / EN

Priorité : MUST ● | 3 pts

En tant que **Visiteur ou Membre**

Je veux **naviguer sur le site via un en-tête responsive (menu burger mobile) et basculer la langue entre Français et Anglais**

Afin de **consulter le site dans ma langue sur mobile ou ordinateur**

Critères d'acceptation :

- Page fonctionnelle traduite via de l'API DeepL et i18n.
- Toutes les pages responsives aux formats mobile et tablette.
- Menu burger fonctionnel permettant d'accéder à toutes les rubriques sur les écrans tactiles et mobiles.

US-01.7 | Consultation du Catalogue de Jeux de Société

Priorité : MUST ● | 5 pts

En tant que **Visiteur**

Je veux **parcourir la ludothèque du bar, filtrer les jeux par catégorie (Stratégie, Ambiance, Famille) et rechercher par mot-clé**

Afin de **découvrir les jeux disponibles à la réservation**

Critères d'acceptation :

- Affichage responsive du catalogue sous forme de cartes d'informations (visuel, titre, catégorie, nombre de joueurs min/max, durée, badge de stock boutique).
- Barre de recherche fonctionnelle permettant de filtrer instantanément par mot-clé (titre du jeu).
- Boutons de filtres par catégorie (Stratégie, Ambiance, Famille, Tous) permettant de restreindre la liste en un clic.

- Bouton d'extension "Voir la description" pour lire le résumé complet sans quitter la page.
- Bouton "Réserver ce jeu" redirigeant vers le formulaire de réservation avec le champ pré-rempli.

US-01.8 | Consultation de l'Agenda et Filtres par Jeu TCG

Priorité : MUST ● | 5 pts

En tant que **Joueur**

Je veux **consulter le calendrier complet des tournois et animations groupés par mois et les filtrer par jeu TCG**

Afin de **planifier mes sorties ludiques à l'avance**

Critères d'acceptation :

- Affichage chronologique des tournois et animations organisés et groupés par mois.
- Filtres interactifs par licence / jeu TCG (Magic: The Gathering, Pokémon, Yu-Gi-Oh!, Lorcana, Tous).
- Affichage clair sur chaque carte de la date, l'heure, les frais d'inscription, la jauge de remplissage/places restantes et le statut.
- Bouton d'action adaptatif ("S'inscrire", "Déjà inscrit", "Complet") s'ajustant dynamiquement au statut du joueur.

US-01.9 | Annulation de Réservation par le Membre

Priorité : MUST ● | 3 pts

En tant que **Joueur connecté**

Je veux **annuler une réservation de table depuis la section "Mes Activités" sur mon profil**

Afin de **libérer la table pour d'autres joueurs en cas d'empêchement**

Critères d'acceptation :

- Affichage de la liste des réservations actives dans l'onglet "Mes Activités" du profil utilisateur.
- Bouton "Annuler" présent sur chaque réservation à venir avec demande de confirmation préalable.
- Mise à jour du statut en base de données SQL et libération immédiate du créneau et de la table sans rechargement nécessaire.
- Notification Toast de confirmation affichée à l'utilisateur ("Réservation annulée avec succès").

US-01.11 | Consultation de la Page À Propos & Agréments TCG

Priorité : MUST ● | 2 pts

En tant que **Visiteur**

Je veux **découvrir l'histoire du café-boutique Cicados, nos espaces de jeu et nos accréditations officielles TCG**

Afin de **vérifier la légitimité et l'ambiance du lieu**

Critères d'acceptation :

- Présentation détaillée du concept du café-boutique, des espaces de jeu et des horaires d'ouverture.
- Section dédiée exposant les accréditations officielles des éditeurs TCG (WPN / Wizards Play Network, Pokémon Play!, Konami OTS).
- Intégration des informations pratiques d'accès (adresse à Paris, accès transports).
- Interface 100% responsive adaptée aux formats mobile, tablette et ordinateur.

US-01.12 | Gestion des Membres par l'Administration

Priorité : MUST ● | 5 pts

En tant qu'**Administrateur**

Je veux **consulter la liste des membres inscrits, modifier leurs rôles (USER / ADMIN) et supprimer les comptes obsolètes**

Afin de **modérer la communauté**

Critères d'acceptation :

- Tableau synthétique affichant la liste complète des comptes inscrits (nom, prénom, pseudo, email, rôle actuel).
- Sélecteur/Bouton interactif permettant de basculer dynamiquement le rôle d'un membre entre USER et ADMIN.
- Bouton "Supprimer" avec confirmation pour supprimer un compte obsolète et purger ses données associées (réservations, inscriptions).
- Accès au panneau d'administration strictly restreint aux seuls comptes possédant le rôle ADMIN.

US-01.13 | Inscription et Désinscription aux Animations Gratuites

Priorité : MUST ● | 3 pts

En tant que **Joueur connecté**

Je veux **m'inscrire aux initiations gratuites, soirées à thème et avant-premières depuis la page Événements**

Afin de **réserver ma place aux animations sans frais**

Critères d'acceptation :

- Identification claire des animations et initiations gratuites avec mention "Gratuit" ou tarif "0 €".
- Bouton d'inscription directe actif pour les joueurs connectés, incitant à la connexion pour les visiteurs.
- Bouton de désinscription en un clic permettant d'annuler sa participation directement depuis la carte de l'événement.
- Mise à jour instantanée du compteur de participants et de la jauge de places disponibles.

US-01.14 | Visualisation de la Grille des Disponibilités de Tables

Priorité : MUST  | 5 pts

En tant que **Joueur**

Je veux **consulter un tableau des créneaux horaires sur la page de réservation montrant le statut des 4 tables heure par heure**

Afin d'**adapter l'heure de ma réservation en fonction des tables libres**

Critères d'acceptation :

- Tableau/Grille horaire dynamique montrant la disponibilité heure par heure pour les 4 tables physiques de l'établissement.
- Indicateurs de couleur explicites : Vert (disponible / ≥ 2 tables libres), Orange (1 table libre), Rouge (complet / 0 table libre).
- Désactivation automatique du clic sur les créneaux complets (rouges) ou passés.
- Actualisation automatique de la grille lors de la modification de la date sélectionnée.

2. EPIC 02 — SHOULD HAVE (Fonctionnalités Très Importantes V1)

US-02.1 | Messagerie & Support Client Multi-Tours

Priorité : SHOULD  | 8 pts

En tant que **Joueur connecté**

Je veux **poser une question via une modale de contact, recevoir la réponse sur mon profil et poursuivre l'échange par un bouton "Répondre"**

Afin d'**entretenir une discussion suivie avec l'équipe sans passer par email**

Critères d'acceptation :

- Modale de contact accessible sur le site permettant d'envoyer un message au support.
- Historique complet des échanges (questions/réponses) consultable dans l'espace profil du membre.
- Champ de saisie avec bouton "Répondre" sous le fil de discussion pour poursuivre la conversation multi-tours.
- Badge de notification visuel (pastille rouge) signalant la réception d'une nouvelle réponse de l'équipe support.

US-02.2 | Gestion des Stocks de la Boutique (+ / -)

Priorité : SHOULD  | 5 pts

En tant qu'**Administrateur et Client**

Je veux **voir le stock en magasin sur les cartes de jeux, et permettre au gérant**

d'ajuster le stock en un clic via des boutons + / - sur le dashboard

Afin de **maintenir un inventaire exact en temps réel**

Critères d'acceptation :

- Affichage du niveau de stock en boutique ("En stock : X") sur les fiches de jeux côté client.
- Boutons d'incrémentation (+) et de décrémentation (-) sur chaque fiche de jeu dans le Dashboard Admin.
- Mise à jour instantanée du stock en base de données sans rechargement de page.
- Modification automatique de l'indicateur visuel (ex: affichage "Rupture de stock" si la quantité est égale à 0).

US-02.3 | Importation Automatisée BGG Hot (Proxy BoardGameGeek)

Priorité : SHOULD 🟡 | 8 pts

En tant qu'**Administrateur**

Je veux **importer les 100 jeux les plus populaires depuis BoardGameGeek via un Proxy XML vers JSON**

Afin de **fournir un catalogue riche sans saisie manuelle**

Critères d'acceptation :

- Endpoint Backend Proxy consommant l'API XML2 de BoardGameGeek (BGG Hot 100) et la convertissant en JSON.
- Bouton d'action "Importer les jeux BGG Hot 100" présent sur l'interface d'administration.
- Parsing et insertion automatique en base de données SQL des métadonnées (titre, visuel, durée, joueurs min/max, catégorie).
- Notification Toast précisant le nombre de jeux importés avec succès.

US-02.4 | Tolérance aux Pannes SQL & Secours JSON (Failover)

Priorité : SHOULD 🟡 | 5 pts

En tant que **Système**

Je veux **basculer automatiquement sur un fichier de secours JSON si la base SQL est inaccessible**

Afin de **maintenir le catalogue en ligne sans interruption de service**

Critères d'acceptation :

- Structure try/catch capturant les erreurs d'accès à la base MySQL dans les contrôleurs du catalogue.
- Basculement automatique (*failover*) vers le fichier JSON statique local `backend/src/data/boardgame-list.json` en cas d'indisponibilité SQL.
- Distribution des données au client garantissant que la consultation du catalogue reste fonctionnelle sans interruption.
- Journalisation serveur (*logging error*) pour alerter l'administrateur système.

US-02.5 | Téléversement d'Avatars et Visuels (Multer Upload)

Priorité : SHOULD 🟡 | 5 pts

En tant qu'**Utilisateur et Administrateur**

Je veux **téléverser ma photo de profil ou ajouter des affiches d'événements**

Afin de **personnaliser l'interface et illustrer les cartes**

Critères d'acceptation :

- Middleware `multer` configuré au niveau du backend pour gérer l'upload de fichiers images (JPEG, PNG, WEBP) avec contrôle du poids et du format.
- Champ d'upload d'avatar sur le profil utilisateur et champ d'upload d'affiche d'événement sur le formulaire d'administration.
- Enregistrement des fichiers média dans le répertoire public du serveur avec un nom unique (timestamp / UUID).
- Prévisualisation instantanée de l'image sélectionnée avant la validation du formulaire.

US-02.7 | Notification Pastille Rouge Fixe Admin Navbar

Priorité : SHOULD 🟡 | 3 pts

En tant qu'**Administrateur**

Je veux **voir une pastille rouge fixe sur le bouton Admin de la navigation lorsqu'un message client est en attente de réponse**

Afin de **traiter rapidement les demandes des membres**

Critères d'acceptation :

- Contrôle automatique du nombre de messages clients non lus / non traités.
- Affichage d'une pastille d'alerte rouge fixe (badge) sur le bouton "Admin" de la barre de navigation.
- Disparition de la pastille une fois toutes les demandes clients traitées.
- Affichage de la notification d'alerte réservé uniquement aux comptes administrateurs.

US-02.8 | Live Preview des Affiches d'Événements Admin

Priorité : SHOULD 🟡 | 3 pts

En tant qu'**Administrateur**

Je veux **prévisualiser l'affiche d'un tournoi ou d'un événement en direct dans le formulaire de création**

Afin de **contrôler le rendu visuel avant la publication**

Critères d'acceptation :

- Composant de prévisualisation en direct (*Live Preview*) affiché en parallèle du formulaire de création/édition d'événement.
- Mise à jour dynamique de l'aperçu au fur et à mesure de la saisie (titre, jeu, date, tarif, visuel).
- Rendu identique à la carte d'événement finale affichée aux membres.

● 3. EPIC 03 — COULD HAVE (Fonctionnalités Optionnelles / Confort V1.5)

US-03.1 | Paiement en Ligne des Inscriptions de Tournois (Stripe)

Priorité : COULD ● | 8 pts

En tant que **Joueur**

Je veux **régler mes frais d'inscription aux tournois par carte bancaire lors de mon inscription en ligne**

Afin de **valider définitivement ma réservation sans passer par la caisse du magasin**

Critères d'acceptation :

- Intégration du composant de paiement sécurisé Stripe Checkout / Elements lors de l'inscription à un tournoi payant.
- Validation instantanée de la transaction et confirmation de l'inscription côté serveur via Webhook Stripe.
- Affichage du statut "Inscrit et Payé" sur l'espace utilisateur.
- Annulation automatique de la réservation en cas d'échec ou d'abandon du paiement.

US-03.2 | Notifications Temps Réel WebSockets (Socket.io)

Priorité : COULD ● | 8 pts

En tant que **Joueur et Administrateur**

Je veux **recevoir les notifications de messages et de mises à jour de jauges instantanément sans polling**

Afin d'**avoir un fil d'actualités en direct**

Critères d'acceptation :

- Connexion WebSocket persistante établie via Socket.io entre le client React et le serveur Express.
- Notification instantanée lors de l'envoi d'un message support ou de la modification d'une jauge de tournoi sans rafraîchissement.
- Mise à jour en direct des places restantes visibles par tous les utilisateurs connectés.

US-03.3 | Filtres de Recherche Avancés par Durée et Mécanique

Priorité : COULD ● | 3 pts

En tant que **Joueur**

Je veux **filtrer la ludothèque par durée exacte de partie et mécanique ludique (deckbuilding, draft, pose de tuiles)**

Afin de **trouver le jeu idéal pour ma soirée**

Critères d'acceptation :

- Sélecteur/Curseur de plages de durée (ex: < 30 min, 30-60 min, > 90 min) sur le catalogue de jeux.
- Filtres par tags de mécaniques de jeu (Deckbuilding, Draft, Placement d'ouvriers, Pose de tuiles).
- Combinaison possible des filtres de durée, de mécanique et de recherche textuelle.

US-03.4 | Export PDF du Récépissé de Réservation

Priorité : COULD  | 3 pts

En tant que **Joueur**

Je veux **télécharger un ticket PDF récapitulatif de ma réservation de table avec QR Code**

Afin de **le présenter rapidement à l'accueil du café-boutique**

Critères d'acceptation :

- Bouton "Télécharger mon récépissé PDF" disponible sur chaque réservation validée dans l'espace membre.
- Génération dynamique d'un document PDF récapitulatif (date, heure, table, nom du jeu, QR Code de validation).
- Fichier téléchargé prêt à l'impression ou à la présentation sur smartphone à l'accueil du bar.

 4. EPIC 04 — WON'T HAVE (Fonctionnalités Hors Périmètre V1 / Futures V2)

US-04.1 | Application Mobile Native (React Native iOS/Android)

Priorité : WON'T  | 13 pts

En tant que **Joueur mobile**

Je veux **installer l'application Cicados depuis l'App Store / Google Play**

Afin d'**accéder à mes réservations en un tap**

Critères d'acceptation :

- [Hors périmètre V1] : Application mobile native dédiée développée en React Native pour iOS et Android.
- [Hors périmètre V1] : Déploiement sur l'App Store et Google Play Store avec notifications Push natives.

US-04.2 | Vente en Ligne E-Commerce avec Livraison à Domicile

Priorité : WON'T  | 13 pts

En tant que **Client distant**

Je veux **acheter des boîtes de jeux de société et boosters TCG en ligne et me les faire livrer chez moi**

Afin d'**acheter sans me déplacer physiquement en boutique**

Critères d'acceptation :

- [Hors périmètre V1] Module e-commerce complet comprenant le panier, le paiement en ligne et le calcul des frais d'expédition.
- [Hors périmètre V1] Gestion des envois de colis et suivi des livraisons depuis le Dashboard Admin.

US-04.3 | Système de Ligue & Classement Elo Annuel

Priorité : WON'T  | 8 pts

En tant que **Joueur compétitif TCG**

Je veux **cumuler des points Elo lors des tournois officiels de la boutique**

Afin de **consulter mon classement dans la ligue annuelle Cicados**

Critères d'acceptation :

- [Hors périmètre V1] Système de calcul et de mise à jour des points Elo des joueurs à l'issue de chaque match officiellement enregistré.
- [Hors périmètre V1] Page de classement général de la ligue annuelle Cicados accessible à la communauté.

US-04.4 | Programme de Fidélité & Cartes Cadeaux Scannables

Priorité : WON'T  | 5 pts

En tant que **Client régulier**

Je veux **accumuler des points de fidélité à chaque réservation ou achat et utiliser des cartes cadeaux**

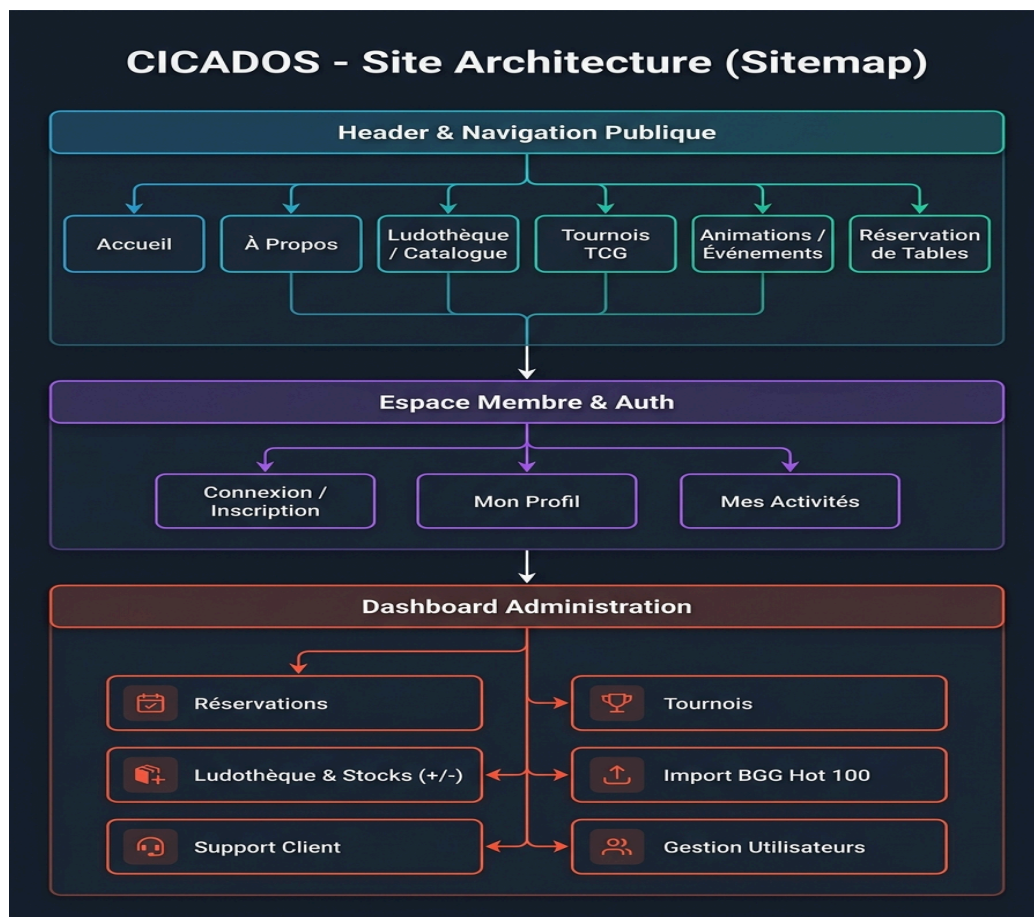
Afin de **bénéficier de réductions sur mes prochaines sessions**

Critères d'acceptation :

- [Hors périmètre V1] Accumulation automatique de points de fidélité sur le compte utilisateur convertibles en avantages/réductions.
 - [Hors périmètre V1] Gestion et scan de cartes cadeaux (virtuelles et physiques) avec QR Code au comptoir de la boutique.
-

3 — CONCEPTION TECHNIQUE

3.1 Arborescence du site



3.2 Dictionnaire des routes de navigation (Front & Back)

Routes Frontend (React Router DOM) :

PUBLIC :

- `/` : Page d'accueil
- `/login` : Formulaire d'authentification utilisateur.
- `/register` : Formulaire d'inscription.
- `/boardgames` : Catalogue de jeux de société.
- `/tournament` : Agenda des tournois et événements.

PRIVÉE :

- `/reservations` : Interface de réservation de table.
- `/my-reservations` : Espace "Mes Activités" pour gérer ses réservations et inscriptions.
- `/profile` : Gestion du profil et Section Mes Messages & Support.

ADMIN :

`/admin` (Rôle ADMIN) : Console d'administration centralisée structurée en 6 onglets (Réservations, Tournois, Événements, Jeux de société, Utilisateurs, Messages).

Routes Backend (Express API REST) — Tableau des Endpoints API REST

(Note : Le tableau des routes Backend au complet est disponible en **Annexe 1** de ce dossier, ci-dessous un échantillon).

1. Authentification & Profil Utilisateur

Méthode	Endpoint / Route API	Accès	Description & Rôle
POST	/api/auth/register	Public	Création d'un nouveau compte utilisateur (hachage du mot de passe avec <code>bcrypt</code>).
POST	/api/auth/login	Public	Authentification de l'utilisateur et génération du token JWT.
PUT	/api/auth/profile	Membre	Mise à jour des coordonnées personnelles du profil membre.

2. Réservations & Disponibilité des Tables

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/reservations	Public	Consultation du tableau des créneaux et de l'état d'occupation des 4 tables pour une date.
GET	/api/reservations/user	Membre	Récupération de l'historique complet des réservations du membre connecté.
DELETE	/api/reservations/:id	Membre	Annulation d'une réservation par le membre et libération immédiate de la table.

3. Tournois TCG & Animations / Événements

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/tournaments	Public	Liste des tournois TCG (Magic, Pokémon, Lorcana, Yu-Gi-Oh!) et jauges de places.
DELETE	/api/tournaments/:id/register	Membre	Désinscription du membre connecté d'un tournoi TCG.
POST	/api/events/:id/register	Membre	Inscription du membre connecté à une animation gratuite.

4. Messagerie & Support Client (Multi-Tours)

Méthode	Endpoint / Route API	Accès	Description & Rôle
POST	/api/messages	Membre	Envoi d'un nouveau message au support client via la modale de contact.
GET	/api/messages/admin	Admin	Récupération de l'ensemble des messages et questions clients (Console Admin).
DELETE	/api/messages/admin/:id	Admin	Supprimer un message ou traiter un fil archivé.

5. Ludothèque & API Proxy BoardGameGeek (BGG)

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/boardgames	Public	Consultation du catalogue de 100+ jeux (avec mécanisme de secours JSON sur panne SQL).
GET	/api/bgg/search	Public/Admin	Proxy XML2 vers JSON pour rechercher des jeux sur l'API externe BoardGameGeek.

GET	/api/bgg/thing	Public/Admin	Récupération des détails techniques d'un jeu spécifique via BoardGameGeek.
------------	----------------	--------------	--

6. Dashboard Administration & Gestion Centralisée

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/admin/reservations	Admin	Affichage de toutes les réservations clients du site.
PUT	/api/admin/events/:id	Admin	Modification des informations d'un événement.
POST	/api/admin/boardgames	Admin	Ajout manuel d'un jeu de société dans le catalogue.

3.3 Modélisation de la Base de Données (MCD, MLD, MPD)

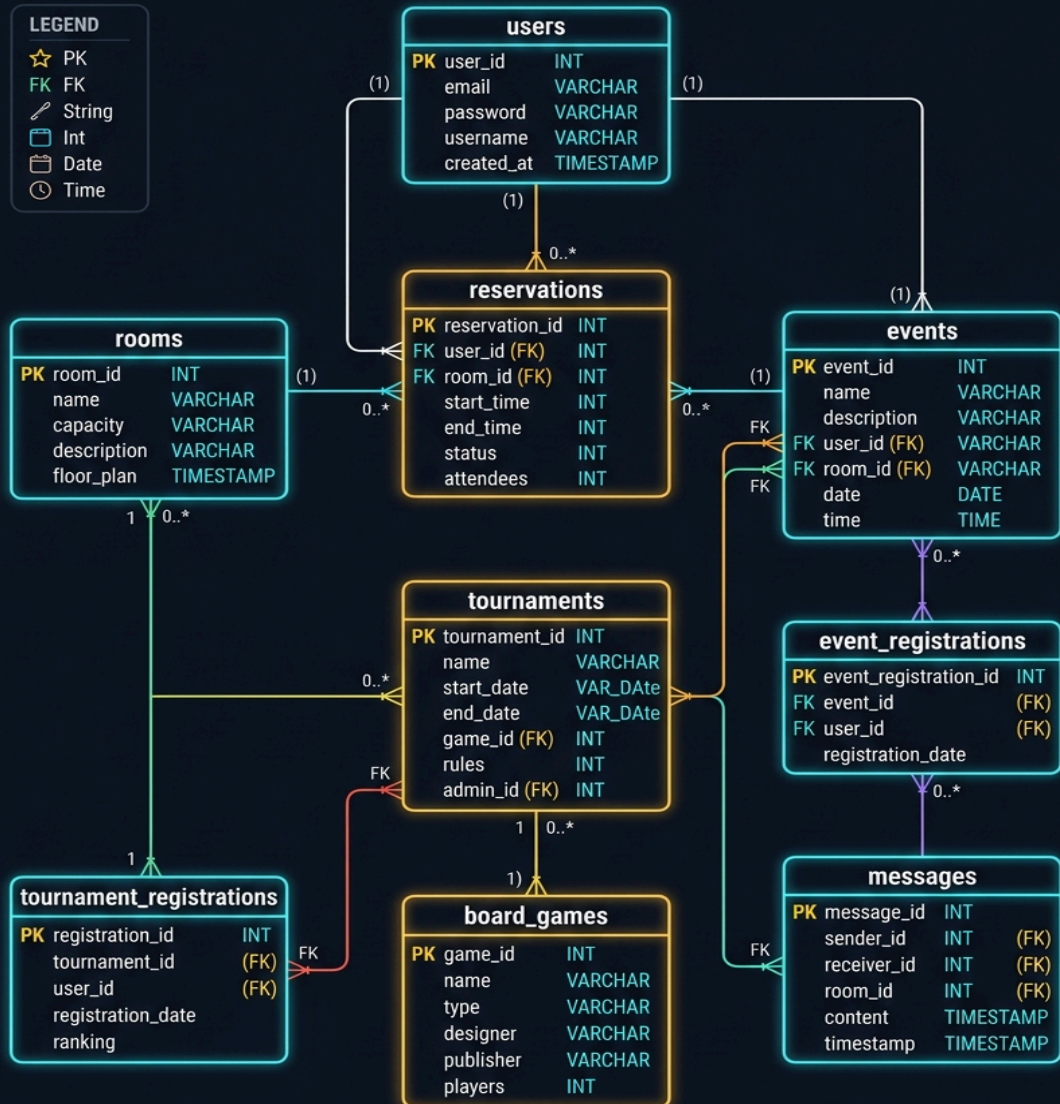
(Note : Le tableau MCD complet est disponible en **Annexe 2** de ce dossier)

(Note : Le script SQL de création complet `schema.sql` est disponible en **Annexe 3** de ce dossier).

1. Modèle Conceptuel de Données (MCD — Schéma Visuel ERD & Entités)

CICADOS: DATABASE SCHEMA DIAGRAM (ERD)

MCD/MLD ENTITY-RELATIONSHIP MODEL



1.1 Entités et Attributs Principaux :

Entité	Attributs Principaux	Rôle / Description
USER	id, email, password, firstname, lastname, pseudo, avatar_url, role	Utilisateurs inscrits (Visiteurs, Membres, Administrateurs).
ROOM	id, name, capacity, description	Tables physiques / espaces de jeu de l'établissement (4 tables).
TOURNAMENT	id, name, game, date, capacity, price, description	Tournois TCG compétitifs organisés par le bar.
EVENT	id, name, type, game, date, capacity, price, description	Animations gratuites (avant-premières, drafts, initiations).

1.2 Associations et Cardinalités :

Association	Entité Source	Cardinalité	Entité Cible	Description de la Relation
Effectuer	USER	(0,N) — (1,1)	RESERVATION	Un utilisateur peut faire 0 à N réservations. Une réservation appartient à 1 seul utilisateur.
Occuper	ROOM	(0,N) — (1,1)	RESERVATION	Une table physique peut être réservée 0 à N fois. Une réservation concerne 1 seule table.
S'inscrire (Tournoi)	USER	(0,N) — (1,1)	TOURNAMENT_ REGISTRATI ON	Un membre peut s'inscrire à 0 ou plusieurs tournois.

2. Modèle Logique de Données (MLD)

users (**id**, email, password, firstname, lastname, pseudo, avatar_url, role, created_at)
rooms (**id**, name, capacity, description, created_at, updated_at)
reservations (**id**, #user_id, #room_id, start_time, end_time, game_type, specific_game, players_count, status, created_at, updated_at)
tournaments (**id**, name, game, date, capacity, price, description, created_at)
tournament_registrations (**id**, #tournament_id, #user_id, registered_at)
board_games (**id**, name, min_players, max_players, play_time, category, description, image_url, rules_url, stock, created_at)
events (**id**, name, type, game, date, capacity, price, description, created_at)
event_registrations (**id**, #event_id, #user_id, registered_at)
messages (**id**, #user_id, subject, content, is_read, admin_reply, replied_at, user_read, created_at)

3. Modèle Physique de Données (MPD — Dictionnaire de Données MySQL)

3.1 Table `users` (Utilisateurs) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant unique de l'utilisateur.
email	VARCHAR(255)	NON	UNIQUE / INDEX	Adresse e-mail unique de connexion.
pseudo	VARCHAR(100)	OUI	-	Pseudonyme public du joueur.
role	ENUM('USER', 'ADMIN')	NON	DEFAULT 'USER'	Rôle applicatif (accès d'administration si ADMIN).

3.2 Table `reservations` (Réservations de Tables) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant unique de la réservation.
user_id	INT UNSIGNED	NON	FOREIGN KEY	Référence au membre (users.id - CASCADE).
room_id	INT UNSIGNED	NON	FOREIGN KEY	Référence à la table physique (rooms.id - CASCADE).
status	ENUM('PENDING', 'CONFIRMED', 'CANCELLED')	NON	DEFAULT 'PENDING'	Statut opérationnel de la réservation.

3.3 Table `messages` (Support Client Multi-Tours) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
user_id	INT UNSIGNED	NON	FOREIGN KEY	Référence à l'expéditeur membre (users.id).
content	TEXT	NON	-	Contenu rédigé par l'utilisateur.
replied_at	DATETIME	OUI	-	Date et heure d'envoi de la réponse admin.

3.4 Table `board\_games` (Ludothèque & Stocks Boutique) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant unique du jeu.
name	VARCHAR(255)	NON	UNIQUE	Nom officiel du jeu de société.
stock	INT UNSIGNED	NON	DEFAULT 1	Quantité en stock magasin (boutons + / -).

3.4 Dictionnaire de données de la base de données

Table : messages (Support Client Multi-Tours) :

Gère les échanges de messagerie et de support entre les membres et l'équipe d'administration.

Champ / Colonne	Type SQL	Spécificités / Contraintes	Rôle & Description Fonctionnelle
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique du message.
user_id	INT UNSIGNED	FOREIGN KEY, ON DELETE CASCADE	Identifiant du membre auteur (users.id).
is_read	TINYINT(1)	NOT NULL, DEFAULT 0	0 = Message non lu par l'admin (badge NOUVEAU), 1 = Lu.

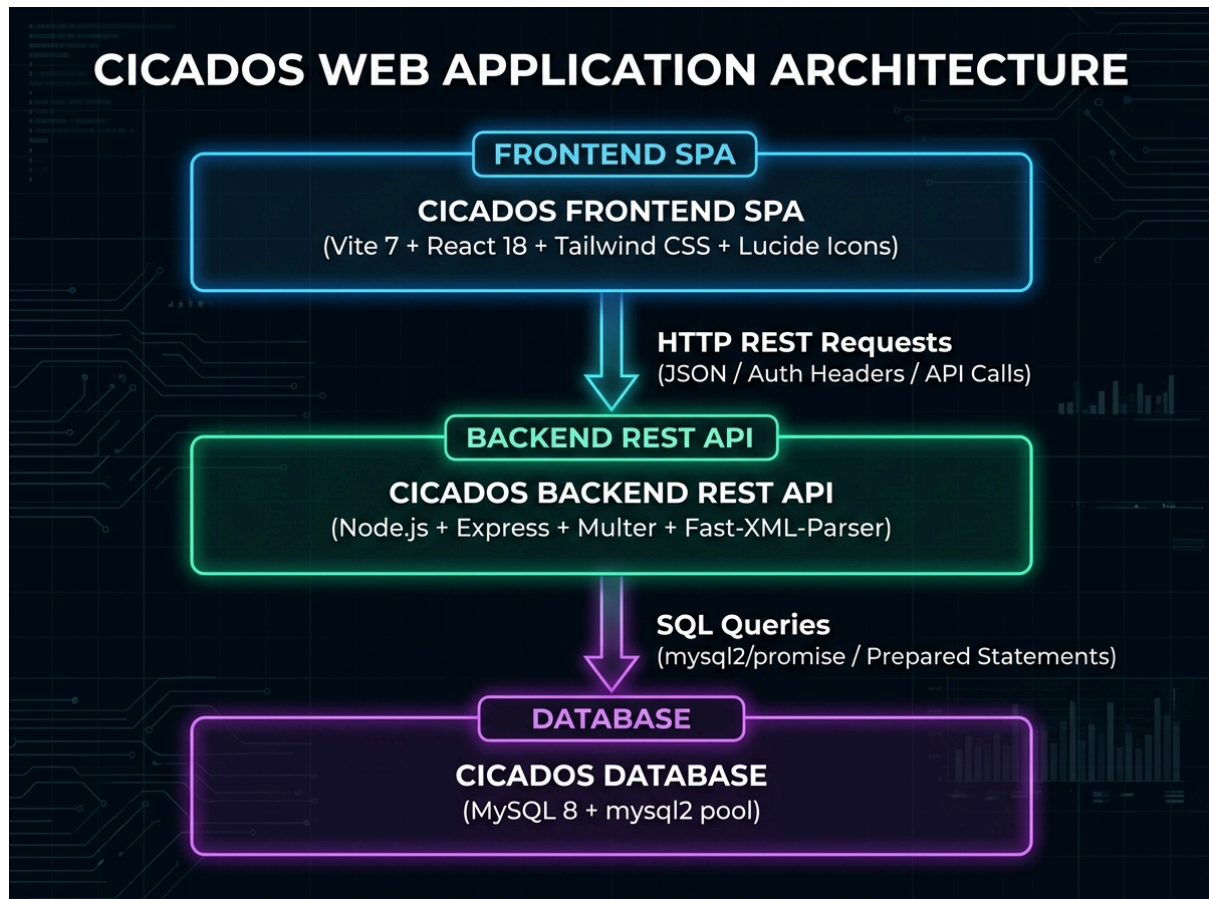
Évolutions des Tables Existantes :

Au cours du développement du projet, la structure de la base de données a naturellement évolué pour s'adapter aux nouveaux besoins fonctionnels et affiner la modélisation des données. Plusieurs tables ont été remaniées et certaines colonnes ont été ajoutées, modifiées ou supprimées afin d'optimiser les requêtes, d'assurer l'intégrité des relations et d'améliorer les performances globales de l'application.

Table Concernée	Colonne Ajoutée	Type SQL	Description & Utilité Fonctionnelle
board_games	stock	INT UNSIGNED, DEFAULT 1	Indique le nombre d'exemplaires du jeu disponibles en magasin, ajustable en 1 clic (+ / -) sur le Dashboard Admin.
users	pseudo	VARCHAR(100), NULL	Pseudonyme public du membre affiché sur les réservations et classements.
users	avatar_url	VARCHAR(500), NULL	Lien vers l'image de profil téléversée par le membre via le middleware <code>Multer</code> .

4 — SPÉCIFICATIONS TECHNIQUES, STACK ET OUTILS UTILISÉS

4.1 Choix et justification de la stack et des outils (Problématique / Besoin / Solution)



React 18 & Vite 7 — Frontend SPA & Dynamic Rendering :

• **Justification de l'architecture SPA :** J'ai choisi une architecture Single Page Application (SPA) plutôt qu'une approche classique Multi-Page (MPA). Sur un site de réservation, j'évite ainsi les rechargements complets de page à chaque filtrage de jeu ou changement de créneau, qui génèrent de la latence et détruisent l'immersion. En maintenant l'état en mémoire et en ne téléchargeant que les données brutes (JSON), ma SPA garantit une réactivité de type application native.

- **React 18** : J'ai privilégié React pour la maturité de son écosystème. J'utilise des fonctionnalités comme `useTransition` pour prioriser les saisies utilisateur (ex. : recherche dynamique de jeux) par rapport aux re-rendus secondaires. Mon approche par composants m'a permis d'isoler la logique métier (ex. : un composant `<TablePicker/>`)
- **Justification de Vite 7** : J'ai opté pour Vite afin d'exploiter les ES Modules (ESM) natifs du navigateur, ce qui me permet d'obtenir un démarrage sous la seconde

Tailwind CSS & Lucide Icons :

- **Justification de Tailwind** : J'ai choisi Tailwind CSS pour éviter la croissance indéfinie des feuilles de style et les risques de régression liés au CSS classique. Tailwind me permet d'encapsuler le style directement dans le JSX. Grâce à son moteur de compilation qui n'extrait que les classes utilisées, je garantis un bundle CSS de taille constante, quelle que soit la taille de mon application.

Lucide Icons : Plutôt que de charger une police d'icônes complète (ex. : Font Awesome) qui forcerait le navigateur à télécharger des centaines d'icônes inutilisées, j'ai choisi Lucide Icons. Comme chaque picto y est exporté sous forme de composant SVG pur, je permets à Vite de n'inclure dans le build que les icônes que j'affiche réellement.

Node.js & Express — Backend API REST Asynchrone :

- **Justification Node JS**: J'ai choisi Node.js pour gérer efficacement les pics de réservation lorsque plusieurs utilisateurs interagissent simultanément avec mon API. Grâce à son architecture événementielle et ses opérations d'E/S asynchrones, mon backend gère de nombreuses connexions concurrentes sans créer systématiquement un thread dédié à chaque requête.
- **Express** : J'ai retenu Express pour sa légèreté et sa transparence, Express m'offre un contrôle total sur le pipeline de middlewares. Ce choix me garantit un code verbeux, mais immédiatement compréhensible et facilement maintenable.
- **Sécurisation par Middleware Pipeline** : Mon traitement des requêtes suit une séquence stricte : filtrage CORS, validation du schéma d'entrée, contrôle d'authentification par token JWT, puis exécution du contrôleur métier. Je capture les erreurs via un middleware centralisé.

MySQL 8.0 & driver mysql2/promise — Persistance & Intégrité Relationnelle :

- **Justification de l'approche Relationnelle** : J'ai écarté les bases NoSQL qui, bien qu'offrant de la flexibilité, échouent à garantir la cohérence stricte indispensable à mon

système de réservation. Mon domaine métier pour le bar à jeux repose sur des relations fortes (un membre réserve une table spécifique, pour un créneau donné, autour d'un jeu précis). J'ai donc choisi MySQL 8.0 pour son moteur InnoDB, garantissant les propriétés ACID et l'intégrité référentielle via des clés étrangères.

- **Prévention du surbooking** : Pour empêcher que deux utilisateurs ne réservent la même table au même millième de seconde, j'isole l'opération dans une transaction SQL (**START TRANSACTION**). Si une seconde requête tente d'insérer le même couple, la base rejette l'opération et déclenche un rollback automatique.

- **Justification du Connection Pooling** : J'utilise le module `mysql2/promise` pour gérer un pool de connexions persistantes réutilisées par l'API Express. Cela m'évite de rouvrir une connexion TCP à la base de données à chaque requête, ce qui coûterait plusieurs dizaines de millisecondes. De plus, le recours systématique aux requêtes préparées me permet de séparer la structure de la requête des données utilisateur, ce qui neutralise nativement 100 % des risques d'injections SQL.

Multer Upload Middleware — Gestion Sécurisée des Médias :

- **Justification de l'interception multipart** : J'ai intégré Multer pour gérer le format `multipart/form-data`, que le serveur Express ne sait pas traiter nativement lors de l'envoi de fichiers. Cela me permet de traiter le flux avant qu'il ne touche le système de fichiers, m'évitant ainsi de stocker un fichier corrompu ou malveillant.

Analyse des risques de sécurité & contre-mesures :

- *Attaque par exécution de code* : Un utilisateur renommant un script `.php` ou `.js` en `.jpg`. **Parade** : Multer inspecte le type réel du fichier et applique une liste blanche stricte (`image/jpeg`, `image/png`, `image/webp`).
- *Attaque par traversée de répertoire* : Injection de séquences `../` dans le nom pour écraser des fichiers système. **Parade** : Le nom d'origine est détruit et remplacé par un identifiant unique.
- *Déni de service (DoS par stockage)* : Envoi de fichiers géants. **Parade** : Une limite matérielle (ex: 2 Mo) est fixée au niveau du middleware.

Fast-XML-Parser — Proxy & Transcodage d'API Externe :

- **Justification du Proxy Backend** : J'ai délégué l'interrogation de l'API BoardGameGeek à mon backend Node.js pour contourner les restrictions CORS du navigateur. L'API BGG renvoyant exclusivement du XML, j'utilise un parser sur le serveur (`fast-xml-parser`) pour convertir le flux, afin de ne distribuer à mon frontend React qu'un objet JSON.

• **Justification du Proxy Backend** : J'ai délégué l'interrogation de l'API BoardGameGeek à mon backend Node.js pour contourner les restrictions CORS du navigateur. L'API BGG renvoyant exclusivement du XML, j'utilise un parser sur le serveur (**fast-xml-parser**) pour convertir le flux, afin de ne distribuer à mon frontend React qu'un objet JSON.

Arbitrage fast-xml-parser : **fast-xml-parser** est reconnu comme le parser XML le plus rapide de l'écosystème Node.js.

Normalisation des données : Il extrait uniquement les champs nécessaires (nom du jeu, image, nombre de joueurs, durée), transforme les structures XML en tableaux JSON.

Puppeteer (Tests E2E) — Automation & Assurance Qualité :

• **Justification des tests E2E** : Des tests unitaires valident qu'une fonction marche de manière isolée, mais ils ne garantissent pas que l'interface React, le backend Express et la BDD MySQL communiquent correctement ensemble. Puppeteer valide la chaîne d'intégration complète en simulant les actions réelles d'un client dans un vrai navigateur.

Arbitrage Puppeteer : Puppeteer, développé par Google, offre un contrôle natif ultra-rapide du moteur Chromium via le protocole DevTools, s'intégrant parfaitement dans des scripts Node.js sans surcouche complexe.

Rentabilité opérationnelle : La suite de tests E2E automatise les parcours critiques (connexion -> recherche de jeu -> réservation d'une table). Une exécution de 30 secondes en mode *headless* remplace 15 minutes de tests manuels répétitifs avant chaque déploiement, éliminant totalement le risque de livrer une régression bloquante en production.

4.3 Gestion de projet Agile (Méthodologie Scrum et sprints)

Le projet Cicados a été conduit selon la méthodologie **Agile Scrum**, découpé en 4 sprints de développement successifs :

Sprint	Objectif Principal	Livrables & Réalisations
Sprint 0	Cadrage & Conception	Cahier des charges, Wireframes Figma, Schéma SQL (<code>schema.sql</code>), Dictionnaire de données.

Sprint 1	Architecture MVC & Core	Serveur Express, pool MySQL, maquette Figma Tailwind, pages Accueil, Catalogue et Détails de jeux.
Sprint 2	Auth & Réservations	Authentification JWT, algorithme anti-surbooking, grille horaire dynamique des 4 tables.
Sprint 3	Admin, Messagerie & Tests	Dashboard Admin 6 onglets, messagerie support multi-tours, gestion des stocks, tests E2E Puppeteer.

4.4 Conventions Git, commits et versioning

Le versioning de l'application est géré sous **Git** avec une convention de nommage des commits stricts (*Standard Conventional Commits*) :

- `feat(...)` : Ajout d'une nouvelle fonctionnalité.
- `fix(...)` : Correction de bug.
- `refactor(...)` : Restructuration du code sans changement de comportement.
- `docs(...)` : Mise à jour de la documentation et du cahier des charges.
- `test(...)` : Ajout et exécution de scripts de tests automatisés Puppeteer.

5 — RÉALISATIONS TECHNIQUES ET ÉTUDE DU CODE SOURCE

Dans cette section, je présente les réalisations logicielles majeures que j'ai conçues et développées pour le projet **Cicados**. J'ai choisi de mettre en avant ces fichiers pour leur forte valeur métier et leur pertinence technique au regard des deux activités-types de mon **Titre Professionnel Développeur Web et Web Mobile (DWWM)** :

- **Activité-Type 1 (AT1)** : Développer la partie front-end d'une application web ou web mobile sécurisée.
- **Activité-Type 2 (AT2)** : Développer la partie back-end d'une application web ou web mobile.

5.1 ÉTUDE DES FICHIERS CLÉS DU FRONT-END (ACTIVITÉ-TYPE 1)

1. Composant de Réservation & Planning Interactif : frontend/src/pages/Reservations.jsx

AT1 : Interface dynamique

- **Mon rôle & objectif métier** : J'ai développé l'interface principale de réservation permettant aux clients de réserver une table de jeu physique (duels TCG ou jeux de société) sur un créneau horaire précis, avec calcul dynamique des disponibilités en direct.
- **Pertinence** : Il démontre ma capacité à concevoir un calendrier interactif personnalisé sans dépendance tierce lourde, à orchestrer plusieurs flux d'API asynchrones en parallèle et à gérer un état réactif complexe en temps réel.

Fonctionnement clé de mon code :

- **Calendrier** : J'ai implémenté un algorithme qui génère la grille mensuelle exacte, gère l'alignement des jours de la semaine et désactive automatiquement les dates passées.
- **Orchestration asynchrone multi-sources** : Dès que l'utilisateur sélectionne une date ou un horaire, j'exécute trois requêtes coordonnées : le chargement des jeux ([/api/boardgames](#)), l'état d'occupation du planning ([/api/reservations?date=...](#)) et la disponibilité du jeu choisi ([/api/reservations/game-availability](#)).
- **Calcul d'occupation côté client ([getOccupiedTablesCount](#))** : J'ai créé une fonction qui filtre les réservations actives afin d'indiquer en direct au client le taux de remplissage des 4 tables pour chaque créneau horaire, sans solliciter inutilement le serveur.

```
// Extrait : Mon calcul du taux d'occupation par tranche horaire dans Reservations.jsx
const getOccupiedTablesCount = (slotTime) => {
  if (!formData.date || !existingReservations.length) return 0;

  const slotDateTime = new Date(`${formData.date} ${slotTime}:00`);

  return existingReservations.filter(res => {
    const start = new Date(res.start_time);
    const end = new Date(res.end_time);
    // Je vérifie si l'heure analysée est comprise dans une réservation existante
    return slotDateTime >= start && slotDateTime < end;
  }).length;
};
```

2. Gestion des Tournois TCG & Inscriptions Réactives : frontend/src/pages/Tournaments.jsx

AT1 : Asynchronisme & État

- **Mon rôle & objectif métier** : J'ai conçu la page de consultation des tournois officiels intégrant les jauges de places restantes et l'inscription/désinscription instantanée.
- **Pertinence** : J'y illustre ma maîtrise de la gestion d'état asynchrone avancée avec actions conditionnelles, feedback utilisateur instantané et isolation du chargement par carte de tournoi.

Fonctionnement clé de mon code :

- **Action unifiée (`handleAction`)** : J'ai factorisé la logique pour détecter si le joueur est déjà inscrit afin de basculer dynamiquement entre un appel HTTP POST (inscription) ou DELETE (désinscription), tout en redirigeant vers `/login` si l'utilisateur est non authentifié.
- **Isolation de l'état de chargement (`actionLoadingId`)** : J'isole l'indicateur de chargement uniquement sur le bouton du tournoi cliqué, ce qui évite de bloquer l'interaction sur les autres cartes.
- **Accordéon réactif des participants (`toggleParticipants`)** : J'ai développé un affichage accordéon permettant de consulter instantanément la liste nominative des joueurs inscrits avec leurs avatars sans recharger l'arbre des composants.

```
// Extrait réel : frontend/src/pages/Tournaments.jsx (lignes 89 à 118)
const handleAction = async (tourneyId, isRegistered) => {
  if (!isAuthenticated) {
    navigate('/login');
    return;
  }

  setActionLoadingId(tourneyId);

  try {
    const token = localStorage.getItem('token');
    const url = `http://localhost:5050/api/tournaments/${tourneyId}/register`;
    const method = isRegistered ? 'DELETE' : 'POST';

    const response = await fetch(url, {
      method,
      headers: { 'Authorization': `Bearer ${token}` }
    });

    const data = await response.json();

    if (response.ok) {
      // Mise à jour réactive locale de mon état sans recharger la page
      setMyRegistrations(prev =>
        isRegistered
          ? prev.filter(id => id !== tourneyId)
          : [...prev, tourneyId]
      );
      // Mise à jour dynamique de la liste des participants
      setTournaments(prevTournaments =>
        prevTournaments.map(t => t.id === tourneyId ? { ...t, ... } : t)
      );
    }
  } finally {
    setActionLoadingId(null);
  }
};
```

3. Console d'Administration & Back-Office : frontend/src/pages/DashboardAdmin.jsx

AT1 : Sécurisation & Tri

- **Mon rôle & objectif métier** : J'ai développé un tableau de bord complet permettant aux administrateurs de piloter l'ensemble des données de la plateforme (Réservations, Tournois, Événements, Jeux en ludothèque, Utilisateurs et Support).
- **Pertinence** : Il prouve ma capacité à structurer un back-office à fort volume de données, à implémenter un moteur de tri dynamique multi-colonnes et à sécuriser l'accès client selon les rôles (RBAC).

Fonctionnement clé de mon code :

- **Protection d'accès côté client** : Je contrôle le rôle utilisateur stocké dans le jeton dès le montage du composant : si le profil n'est pas ADMIN, je déclenche une redirection immédiate vers la page d'accueil.
- **Moteur de tri dynamique (`handleSort`)** : J'ai créé un gestionnaire permettant de réordonner les tables de données par date, capacité, client ou statut en cliquant sur les en-têtes de colonnes.
- **Modales de confirmation (`deleteConfirmId`)** : J'ai sécurisé toutes les opérations destructives en imposant une confirmation explicite avant d'émettre la requête HTTP DELETE.

```
// Extrait réel : frontend/src/pages/DashboardAdmin.jsx (lignes 39 à 46 et 925 à 952)
// 1. Inversion du tri au clic sur une colonne de tableau
const handleSort = (key) => {
  if (resSortKey === key) {
    setResSortOrder(prev => prev === 'asc' ? 'desc' : 'asc');
  } else {
    setResSortKey(key);
    setResSortOrder('asc');
  }
};

// 2. Comparateur dynamique selon le type de données (dates, noms ou statuts)
const sortedReservations = filteredReservations.sort((a, b) => {
  if (!resSortKey) return 0;

  let valA = '', valB = '';
  if (resSortKey === 'start_time') {
    valA = new Date(a.start_time).getTime();
    valB = new Date(b.start_time).getTime();
  } else if (resSortKey === 'user') {
    valA = `${a.firstname || ''} ${a.lastname || ''}`.toLowerCase().trim();
    valB = `${b.firstname || ''} ${b.lastname || ''}`.toLowerCase().trim();
  }

  if (valA < valB) return resSortOrder === 'asc' ? -1 : 1;
  if (valA > valB) return resSortOrder === 'asc' ? 1 : -1;
  return 0;
});
```

5.2 ÉTUDE DES FICHIERS CLÉS DU BACK-END (ACTIVITÉ-TYPE 2)

1. Algorithme d'Overlap Temporel & Allocation :

`backend/src/models/reservation.model.js`

AT2 : Requêtes SQL & Logique métier

• **Mon rôle & objectif métier** : J'ai conçu l'algorithme central qui empêche tout surbooking sur les 4 tables physiques du café et qui garantit qu'un jeu de société à exemplaire unique ne puisse pas être réservé deux fois en même temps.

• **Pertinence** : C'est la pièce maîtresse technique de mon projet. Il démontre ma maîtrise des requêtes SQL préparées, de la gestion mathématique des intervalles temporels et de la sécurité transactionnelle.

Fonctionnement clé de mon code :

- **Calcul précis de l'intervalle** : Je calcule dynamiquement la date et l'heure de fin via `endTime = startTime + duration`.
- **Formule mathématique de détection de chevauchement** :
Condition de conflit : `start_time < :newEndTime AND end_time > :newStartTime AND status != 'CANCELLED'`
- **Allocation dynamique de table physique** : J'extrais l'ensemble des tables disponibles et j'assigne automatiquement la première table libre. Si toutes les tables sont occupées, je lève une exception qui renvoie un code HTTP 400.
- **Contrôle du stock unique (Stock = 1)** : Lorsqu'un client sélectionne un jeu spécifique, je vérifie qu'aucune réservation active n'emprunte cet exemplaire sur la plage horaire souhaitée.
- **Sécurité transactionnelle** : Dès qu'une contrainte échoue, j'interromps le script immédiatement avec un `throw new Error(...)` : aucune ligne n'est insérée en base de données.

```
// Extrait : Ma détection des tables occupées et mon allocation dans reservation.model.js
const sqlOccupied = `
  SELECT room_id FROM reservations
  WHERE start_time < ? AND end_time > ? AND status != 'CANCELLED'
`;
const occupiedRooms = await query(sqlOccupied, [endTime, startTime]);
const occupiedIds = occupiedRooms.map(r => r.room_id);

// J'attribue la première table physique disponible
const availableRoom = rooms.find(r => !occupiedIds.includes(r.id));
if (!availableRoom) {
  throw new Error("Toutes les tables de jeux sont complètes pour ce créneau horaire.");
}
```

2. Sécurité JWT, Hashage Bcrypt & Middlewares RBAC : auth.controller.js & auth.middleware.js

AT2 : Sécurité & OWASP

- **Mon rôle & objectif métier** : J'ai mis en place l'authentification sécurisée des utilisateurs, la délivrance des jetons signés et la protection des points d'accès de mon API REST.
- **Pertinence** : J'y applique strictement les recommandations du référentiel DWSM et de l'OWASP : chiffrement irréversible des mots de passe, tokens temporisés et contrôle d'accès basé sur les rôles (RBAC).

Fonctionnement clé de mon code :

- **Hashage Bcrypt** : Je sale et je hache les mots de passe avec un coût de 10 avant tout enregistrement en base SQL. Aucun mot de passe en clair n'est stocké.
- **Signature numérique JWT** : Lors de la connexion, je délivre un jeton JWT contenant l'identifiant et le rôle de l'utilisateur, signé avec une clé secrète et valide 24 heures.
- **Pipeline Middleware (authMiddleware)** : J'extrais le jeton dans l'en-tête Authorization: Bearer, je valide la signature via `jwt.verify()` et j'injecte l'utilisateur dans l'objet requête `req.user`.
- **Contrôle RBAC (adminMiddleware)** : Je vérifie la présence du rôle ADMIN et je bloque toute tentative non autorisée avec un statut HTTP standard 403 Forbidden.

```
// Extrait : Mon middleware d'authentification JWT (backend/src/middlewares/auth.middleware.js)
const authMiddleware = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Token manquant' });
    }

    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);

    if (!user) return res.status(401).json({ error: 'Utilisateur non trouvé' });

    req.user = user; // J'injecte l'utilisateur validé dans la requête
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Token invalide ou expiré' });
  }
};
```

3. Gestion de Concurrency & Intégrité SQL : backend/src/controllers/tournament.controller.js

AT2 : Intégrité BDD

- **Mon rôle & objectif métier** : J'ai programmé la gestion des inscriptions aux tournois TCG en assurant le respect des jauges de places et l'interdiction stricte des doublons.

• **Pertinence** : Il montre ma maîtrise de l'intégrité relationnelle SQL et ma gestion des exceptions natives du moteur MySQL (ER_DUP_ENTRY).

Fonctionnement clé de mon code :

- **Contrôle des jauges de capacité** : Je vérifie avant toute écriture que `registeredCount < capacity`, et je renvoie un code HTTP 400 explicite si le quota est atteint.
- **Clé composite & Interception d'erreur SQL** : Ma table `tournament_participants` s'appuie sur une clé primaire composite (`tournament_id, user_id`). Si un utilisateur tente de s'inscrire deux fois, le moteur MySQL génère l'erreur ER_DUP_ENTRY que j'intercepte dans le bloc catch pour informer proprement le client sans planter le serveur.

```
// Extrait : Mon inscription sécurisée avec capture du doublon SQL (tournament.controller.js)
export const registerForTournament = async (req, res) => {
  try {
    const tournamentId = req.params.id;
    const userId = req.user.id;

    const tourneyInfo = await Tournament.getCapacityAndCount(tournamentId);
    if (tourneyInfo.registeredCount >= tourneyInfo.capacity) {
      return res.status(400).json({ error: "Ce tournoi est complet." });
    }

    await Tournament.register(tournamentId, userId);
    res.status(201).json({ message: "Inscription réussie !" });
  } catch (error) {
    if (error.code === 'ER_DUP_ENTRY') {
      return res.status(400).json({ error: "Vous êtes déjà inscrit à ce tournoi." });
    }
    res.status(500).json({ error: "Erreur serveur lors de l'inscription." });
  }
};
```

4. Proxy API Externe & Résilience : bgg.controller.js & boardgame.controller.js

AT2 : Résilience & APIs

• **Mon rôle & objectif métier** : J'ai développé un proxy serveur pour importer les données officielles de BoardGameGeek (BGG) et j'ai implémenté un système de secours assurant la continuité de service en cas de panne de la base de données.

• **Pertinence** : J'y résous deux problématiques d'architecture concrètes : le contournement des restrictions de sécurité CORS du navigateur et la mise en œuvre de la Haute Disponibilité.

Fonctionnement clé de mon code :

- **Proxy Edge (bgg.controller.js)** : J'exécute la requête HTTP depuis mon backend Node.js vers l'API XML2 de BGG pour contourner la politique CORS, puis je convertis l'arbre XML en JSON exploitable grâce à `fast-xml-parser`.

- **Mécanisme de basculement de secours (Failover) :** Dans `boardgame.controller.js`, si ma base MySQL ne répond pas, mon bloc catch prend le relais de façon transparente en lisant de manière asynchrone le fichier local `boardgame-list.json`. L'utilisateur continue de naviguer dans le catalogue sans jamais subir d'écran blanc.

```
// Extrait réel : backend/src/controllers/boardgame.controller.js (lignes 10 à 42)
export const getBoardGames = async (req, res) => {
  try {
    const games = await BoardGame.findAll();
    res.json(games);
  } catch (error) {
    console.error("Erreur récupération BDD, tentative de basculement sur le fichier JSON local :",
error);
    try {
      const jsonPath = path.join(__dirname, '..', 'data', 'boardgame-list.json');
      const data = await fs.readFile(jsonPath, 'utf8');
      const gamesList = JSON.parse(data);

      // Je mappe les données de secours avec des valeurs par défaut compatibles avec l'UI
      const fallbackGames = gamesList.map((game, index) => ({
        id: index + 10000, // IDs virtuels élevés
        name: game.name,
        min_players: 2,
        max_players: 4,
        play_time: 45,
        category: "Famille",
        description: `Jeu de société : ${game.name}. (Données de secours chargées depuis le
fichier local).`
      }));

      res.json(fallbackGames);
    } catch (jsonError) {
      res.status(500).json({ error: "Impossible de charger les jeux (panne BDD et secours
indisponible)." });
    }
  }
};
```

6 — FOCUS ALGORITHMIQUE & FONCTIONNALITÉS

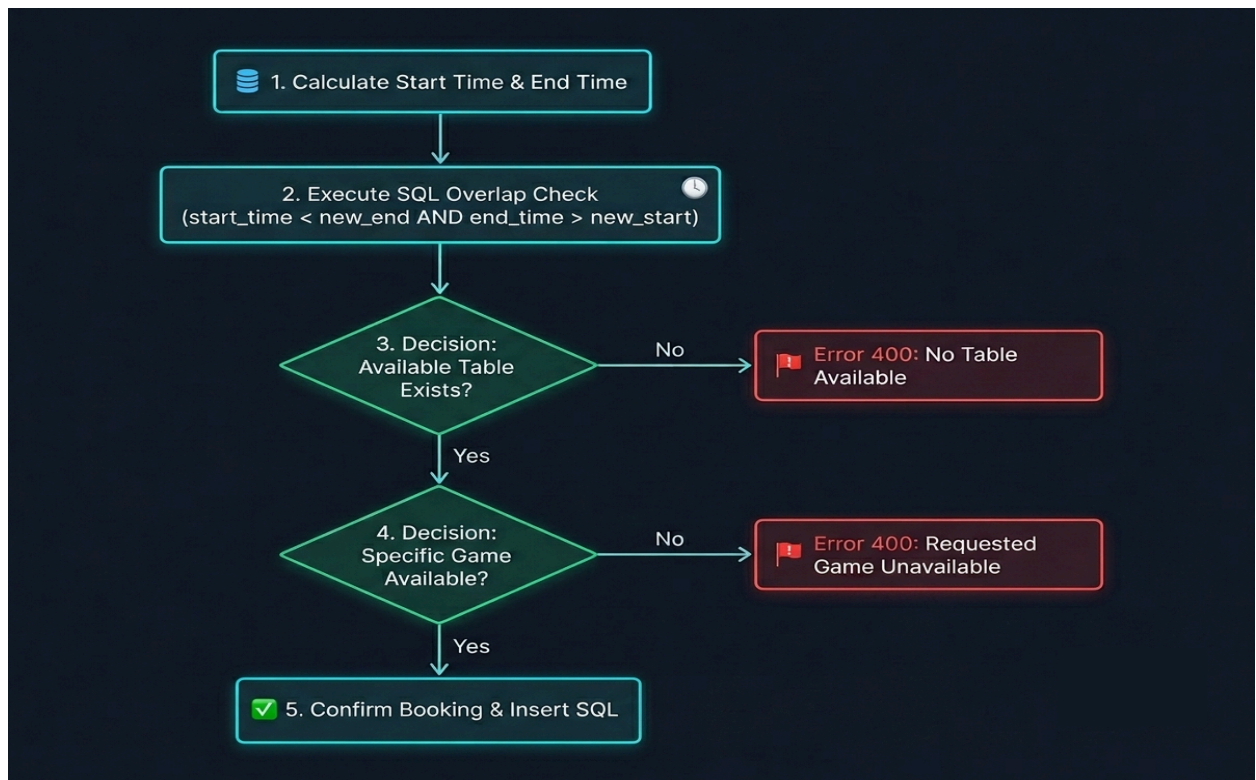
6.1 L'algorithme de détection des conflits de réservation & d'allocation dynamique

L'algorithme de réservation repose sur le principe de **détection de chevauchement d'intervalles temporels** combiné à une **allocation dynamique de tables physiques** et à une **vérification de l'unicité du stock de jeu**.

1. Démonstration Mathématique de Chevauchement Temporel :

Condition logique d'intersection d'intervalles temporels :

Condition de Conflit SQL : `start_time < :new_end_time AND end_time > :new_start_time`



2. Pseudo-code Fonctionnel (Découpage Étape par Étape) :

```

Algorithm_reserveTable.pseudo

// Fonction pour réserver une table dans un restaurant
1  FONCTION reserveTable(Client, Table, DateHeure)
2  // Vérifier la disponibilité de la table
3  SI (Table.estDisponible(DateHeure))
4  // Vérifier le statut de réservation du client
5  SI (Client.peutReserver())
6  // Créer la réservation
7  Reservation = {
8    "client": Client.ID,
9    "table": Table.ID,
10   "heure": DateHeure
11  }
12  // Confirmer la réservation
13  EXECUTER_SQL("INSERT INTO Reservations VALUES (%s, %s, %s)",
    (Reservation.client, Reservation.table, Reservation.heure))
14  RETOURNER "Reservation CONFIRMÉE pour " + Client.Nom
15  FIN SI
16  // Si le client ne peut pas réserver
17  RETOURNER "ERREUR: Client inéligible pour la réservation."
18  SINON
19  // Si la table est déjà réservée
20  RETOURNER "ERREUR: La table #" + Table.ID + " n'est pas DISPONIBLE."
21  FIN SI
22  FIN FONCTION
  
```

Étape	Instruction Algorithmique & Code	Logique Métier & Objectif
Signature	<pre> FONCTION reserverTable(userId, date, heureDebut, dureeHeures, jeuSpecifique, nbJoueurs) </pre>	Point d'entrée principal de la création de réservation.
Étape 1 Calcul Temporel	<pre> T_start = concatener(date, heureDebut) T_end = T_start + dureeHeures (en heures) </pre>	Définit la plage horaire exacte [T_start, T_end] de la session.
Étape 2 Détection Overlap SQL	<pre> tablesOccupeesIds = EXECUTER_SQL("SELECT room_id FROM reservations WHERE start_time < T_end AND end_time > T_start AND status != 'CANCELLED'") </pre>	Identifie toutes les tables physiques déjà réservées sur la même période via la formule d'overlap.
Étape 3 Allocation Table Libre	<pre> toutesLesTables = EXECUTER_SQL("SELECT id FROM rooms") tableLibre = TROUVER(toutesLesTables OU id NOT IN tablesOccupeesIds) SI AUCUNE tableLibre ALORS LEVER_EXCEPTIONS(400, "Toutes les tables sont complètes.") FIN SI </pre>	Assigne la première des 4 tables libres. Rejette en Erreur 400 si l'établissement est complet.
Étape 4 Stock Jeu Unique (Stock = 1)	<pre> SI jeuSpecifique EST NON NULL ALORS jeuReserve = EXECUTER_SQL(...) SI jeuReserve EXISTE ALORS LEVER_EXCEPTIONS(400, "Jeu déjà réservé.") FIN SI FIN SI </pre>	S'assure que le jeu de société demandé n'est pas déjà emprunté sur le même créneau (1 seul exemplaire disponible).
Étape 5 Confirmation SQL	<pre> resId = EXECUTER_SQL("INSERT INTO reservations ... VALUES (...)") RETOURNER resId </pre>	Persiste la réservation validée avec statut CONFIRMED et retourne l'identifiant.

6.2 Le Proxy BGG, l'importation de masse et la conversion XML vers JSON

Pour contourner les restrictions CORS et la structure XML2 complexe renvoyée par l'API officielle de BoardGameGeek, le backend Cicados embarque un contrôleur Proxy. Le

serveur Express interroge l'API XML2 BGG, convertit le flux en JSON natif via `fast-xml-parser` et insère directement les jeux les plus populaires en base de données SQL.

6.3 Le basculement dynamique en cas de panne (Sauvegarde JSON)

En cas d'échec de connexion avec MySQL, le contrôleur du catalogue utilise automatiquement le fichier JSON local `boardgame-list.json`. Ce mécanisme de secours permet de conserver l'affichage du catalogue malgré une indisponibilité temporaire de la base. Il s'agit d'un failover applicatif ciblé, et non d'une architecture de haute disponibilité complète.

6.4 Le Système de Messagerie Multi-Tours & Synchronisation Réactive Temps Réel

Lorsqu'un membre relance une discussion, le contrôleur backend `userReplyMessage` réinitialise `admin_reply` à `NULL` et définit `is_read = 0`. Le frontend émet l'événement natif `messages_updated` pour rafraîchir dynamiquement la pastille rouge du Header et du Dashboard Admin.

Ce mécanisme apporte une réactivité côté client, mais ne synchronise pas plusieurs navigateurs. Le passage à WebSocket / Socket.io est donc identifié comme évolution V2.

7 — JEUX D'ESSAI ET SCÉNARIOS DE VALIDATION

Afin d'assurer la fiabilité des parcours utilisateurs critiques et de **réduire fortement le risque de régression** avant la livraison, j'ai formalisé et exécuté une matrice de recette rigoureuse couvrant les 3 scénarios métier principaux ainsi que l'automatisation E2E :

Test Réalisé	Conditions & Entrées	Résultat Attendu	Résultat Obtenu	Statut
Test 1.1 : Réservation standard Scénario Réservation	Membre authentifié, table disponible, créneau 15h00 - 17h00, jeu <i>Catan</i> choisi.	Création de la réservation, jauge 3/4 restante, toast vert de confirmation.	Écriture SQL (HTTP 201), jauge mise à jour, toast affiché, visible dans le profil.	✓

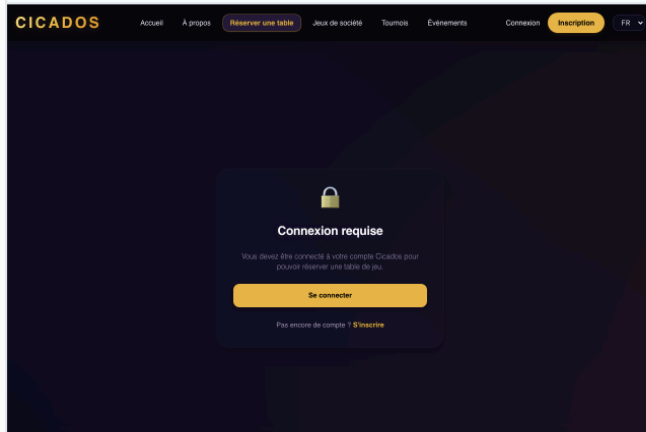
Test 1.2 : Prévention du surbooking Scénario Réservation	4 tables réservées sur 19h00 - 21h00, tentative d'une 5ème réservation.	Blocage immédiat par le backend, rejet transaction, alerte « <i>Aucune table disponible</i> ».	Erreur HTTP 400 renvoyée, aucune écriture parasite en BDD, notification claire.	
Test 2.1 : Inscription tournoi TCG Scénario Tournoi	Tournoi <i>Yu-Gi-Oh!</i> (7/8 inscrits). Joueur connecté clique sur « S'inscrire ».	Jauge passe à 8/8 (Complet), bouton bascule en « Se désinscrire », avatar affiché.	Ligne ajoutée en BDD, mise à jour instantanée de l'état React, jauge saturée.	
Test 2.2 : Garde de saturation Scénario Tournoi	Tournoi complet (8/8). Joueur tente une injection d'inscription via API.	Rejet par <code>tournament.controller.js</code> : « <i>Capacité maximale atteinte</i> ».	HTTP 400 bloquant, intégrité du quota de 8 joueurs strictement préservée.	
Test 2.3 : Désistement libre Scénario Tournoi	Joueur inscrit clique sur « Se désinscrire ».	Place libérée, jauge repasse à 7/8, statut redevient « Disponible ».	Suppression SQL, interface mise à jour en temps réel sans rafraîchissement.	
Test 3.1 : Envoi message membre Scénario Messagerie	Membre soumet : « <i>Puis-je apporter mon propre tapis ?</i> » depuis son profil.	Persistance avec <code>is_read = 0</code> , badge NOUVEAU et pastille rouge dans le Header Admin.	Écriture SQL conforme, événement custom déclenché, compteur incrémenté.	
Test 3.2 : Réponse admin Scénario Messagerie	Admin valide : « <i>Oui tout à fait !</i> ».	Marqué lu par l'admin, réponse rattachée, pastille sur l'avatar du membre.	Notification active sur l'avatar du membre, réponse lisible dans son profil.	
Test 3.3 : Relance multi-tours Scénario Messagerie	Membre clique « Répondre » : « <i>Merci ! Et pour les dés ?</i> ».	Fil incrémenté, bascule automatique en NOUVEAU côté admin avec historique.	Historique préservé, alerte réactivée côté admin, traçabilité complète.	
Test 4.1 : Test E2E Puppeteer Automatisation	Script headless : Login → Choix créneau → Réservation → Contrôle Toast.	Parcours complet exécuté en moins de 35s, screenshot de confirmation généré.	Exécution réussie en 28s, capture produite, 100% des assertions passées.	

8 — Captures Visuelles

Ces captures annotées permettent d'appréhender instantanément les parcours utilisateurs clés :

« Voici la réservation côté utilisateur

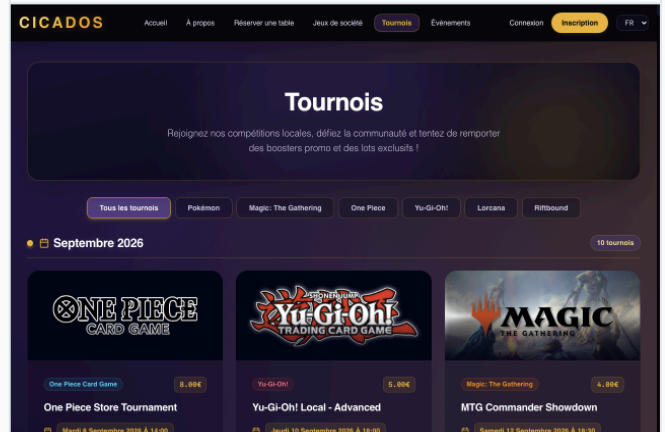
»



Sélection visuelle de la date, du créneau horaire et du jeu de société. Retour immédiat sur la disponibilité des 4 tables et toast de validation.

« Voici l'agenda des tournois TCG

»



Cartes des tournois officiels avec compte à rebours, jauges de capacité (8 places max), liste des participants avec avatars et bouton d'action dynamique.

« Voici la ludothèque et catalogue »



Fiches de jeux synchronisées avec BoardGameGeek, filtres par catégorie, temps de jeu et complexité, avec basculement de secours sur fichier JSON local.

« Voici l'accueil et la charte graphique »



Identité visuelle immersive jaune/violette (#F4AF23 / #05040a), responsive design et mise en avant des services du café-boutique.

9 — CONFORMITÉ RGPD & PROTECTION DES DONNÉES PERSONNELLES

Dans le cadre de la réalisation de la plateforme **Cicados**, j'ai intégré dès la phase initiale de cadrage (Sprint 0) les exigences réglementaires européennes du **Règlement Général sur la Protection des Données (RGPD - Règlement UE 2016/679)** ainsi que les recommandations de la **CNIL**. La protection de la vie privée des utilisateurs et la souveraineté de leurs données ont constitué un critère d'architecture majeur de mon projet.

9.1 Démarche « Privacy by Design » & Cadre Légal

J'ai adopté une démarche de **Protection des données dès la conception (Privacy by Design)** et de **Protection par défaut (Privacy by Default)** :

- **Transparence** : L'utilisateur est informé de la finalité exacte de chaque information recueillie lors de son inscription ou de ses réservations de tables.
- **Sécurité Préventive** : Aucun accès anonyme non autorisé n'est toléré sur les ressources personnelles des membres (profil, messages, réservations privées).
- **Proportionnalité** : Les fonctionnalités de réservation, de consultation de la ludothèque et de participation aux tournois TCG ont été conçues pour fonctionner avec le minimum absolu de données identifiantes.

9.2 Principe de Minimisation des Données (Article 5.1.c RGPD)

Conformément au principe de minimisation, je n'enregistre en base de données MySQL que les attributs strictement indispensables à la fourniture des services du café-boutique :

- **Courriel (email)** : Identifiant unique de connexion et vecteur d'acheminement des confirmations.
- **Mot de passe (password)** : Haché de manière irréversible via l'algorithme **Bcrypt** (coût de salage de 10). Aucun mot de passe en clair ne transite durablement ni n'est persisté.
- **Nom & Prénom (firstname, lastname)** : Requis pour l'accueil physique au comptoir du café-boutique.
- **Pseudonyme (pseudo)** : Pseudonyme public modifiable affiché sur les cartes de tournois et dans les listes de participants, garantissant l'anonymat du nom patronymique vis-à-vis des autres joueurs.
- **Photo de profil (avatar_url)** : Téléversement facultatif géré via Multer avec validation stricte du type MIME réel.

🚫 **Données superflues délibérément exclues** : Aucune collecte de numéro de téléphone obligatoire, aucune géolocalisation ou traceur GPS de l'utilisateur, et aucun stockage de coordonnées bancaires en base de données. Pour la feuille de route V2, la gestion des paiements Stripe s'effectuera par tokenisation déléguée conforme au standard de sécurité bancaire PCI-DSS.

9.3 Registre des Traitements et Finalités Légitimes (Articles 6 & 30 RGPD)

J'ai formalisé le registre des activités de traitement de la plateforme selon les finalités opérationnelles :

Finalité du Traitement	Données Traitées	Base Légale (Art. 6)	Destinataires	Durée de Conservation
Gestion de compte & Authentification	Email, nom, prénom, pseudo, hash Bcrypt, avatar	Exécution Contrat (CGU)	L'utilisateur, Administrateurs	Durée d'activité du compte + purge après 24 mois sans connexion
Réservation de tables & Emprunt de jeux	ID utilisateur, date, créneau horaire, table, jeu	Exécution Contrat (Réservation)	Personnel du café, Membre	Archivage 1 an pour historique, puis suppression
Inscriptions aux Tournois TCG & Jauges	ID utilisateur, pseudo public, statut d'inscription	Consentement & Contrat	Organisateurs, Joueurs (pseudo seul visible)	Clôture de l'événement + 6 mois d'historique
Messagerie & Support client multi-tours	ID utilisateur, sujet, messages, réponses admin	Intérêt Légitime (Support)	Équipe support, Membre	Jusqu'à résolution de la demande ou suppression du compte

9.4 Mesures Techniques et Organisationnelles de Sécurité (Article 32 RGPD)

Pour garantir la confidentialité et l'intégrité des données personnelles face aux risques d'accès illégitime ou de fuite, j'ai implémenté plusieurs barrières techniques appuyées par le code source :

1. Hachage Cryptographique Unidirectionnel & Authentification JWT

Dans `backend/src/controllers/auth.controller.js`, je sale et hache les mots de passe avec l'algorithme `bcryptjs` (coût de salage de 10). À la connexion, je délivre un JSON Web Token (JWT) signé en HMAC-SHA256 valide 24 heures :

```
// Extrait réel : backend/src/controllers/auth.controller.js
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

// Génère un token JWT sécurisé et temporisé
const generateToken = (user) => {
  return jwt.sign(
    { id: user.id, email: user.email },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '24h' }
  );
};

// Hachage fort avant insertion SQL (aucun mot de passe en clair persisté)
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);
```

2. Neutralisation des Injections SQL par Requêtes Préparées

Dans backend/src/models/user.model.js, toutes les requêtes sont exécutées via des déclarations préparées avec le marqueur d'emplacement ?. La commande SQL est isolée des valeurs injectées par l'utilisateur :

```
// Extrait réel : backend/src/models/user.model.js
async findByEmail(email) {
  const sql = 'SELECT id, email, password, firstname, lastname, pseudo, role, avatar_url FROM users WHERE email = ?';
  const [rows] = await db.query(sql, [email]); // Placeholder '?' étanche
  return rows[0] || null;
}
```

9.5 Exercice des Droits des Personnes Concernées (Articles 15 à 21 RGPD)

L'architecture de Cicados permet à chaque membre d'exercer ses droits en toute autonomie :

- **Droit d'accès et de rectification (Articles 15 & 16)** : Le membre consulte et modifie ses informations personnelles en direct depuis son profil (Profile.jsx connecté à la route PUT /api/auth/profile).
- **Droit à l'effacement / « Droit à l'oubli » (Article 17)** : L'utilisateur peut supprimer définitivement son compte depuis son profil, ou l'administrateur peut procéder à l'effacement depuis son tableau de bord.

3. Intégrité SQL en Cascade (ON DELETE CASCADE)

Dans backend/schema.sql, la structure relationnelle garantit qu'aucune donnée orpheline ne subsiste lors de la suppression d'un compte utilisateur :

```
-- Extrait réel : backend/schema.sql (Purge automatique Art. 17 RGPD)
CREATE TABLE reservations (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id INT UNSIGNED NOT NULL,
  ...
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
-- Appliqué à l'identique sur tournament_registrations, event_registrations et messages
```

4. Endpoint Dédié au Droit à l'Oubli (DELETE /api/auth/delete-account)

J'ai implémenté un contrôleur dédié permettant au membre connecté de purger l'ensemble de ses données personnelles :

```
// Extrait réel : backend/src/controllers/auth.controller.js
export const deleteAccount = async (req, res) => {
  try {
    const userId = req.user.id;
    await User.delete(userId); // Déclenche le ON DELETE CASCADE dans MySQL
    res.status(200).json({ message: 'Compte et données personnelles purgés avec succès (RGPD).' });
  } catch (error) {
    res.status(500).json({ error: 'Erreur serveur lors de la suppression du compte' });
  }
};
```

9.6 Consentement Explicite & Gestion des Traceurs (Recommandations CNIL)

5. Case à Cocher Obligatoire d'Inscription (Opt-In Article 7 RGPD)

Dans frontend/src/pages/Register.jsx, l'inscription est conditionnée par l'acceptation expresse de la politique de confidentialité :

```
// Extrait réel : frontend/src/pages/Register.jsx (Consentement obligatoire)
<div className="flex items-start gap-3 p-3 rounded-xl bg-[#0c0919]/60 border border-white/5">
  <input
    type="checkbox"
    id="acceptTerms"
    required
    checked={acceptTerms}
    onChange={e => setAcceptTerms(e.target.checked)}
  />
  <label htmlFor="acceptTerms">
    J'ai lu et j'accepte les Conditions Générales & la Politique de Confidentialité (RGPD).
  </label>
</div>
```

6. Politique « Zéro Traceur Tiers » et Cookies CNIL

La plateforme Cicados applique une politique stricte de « **Zéro Traceur Tiers** ». Elle n'embarque aucun outil de pistage publicitaire ou de profilage comportemental externe (ni Google Analytics, ni pixel publicitaire). L'unique donnée persistée côté client est le jeton de session JWT dans le `localStorage` du navigateur. Étant strictement nécessaire à la fourniture du service d'authentification expressément sollicité par l'internaute, ce traceur technique est **exempté de recueil préalable de consentement** par bandeau intrusif, conformément aux lignes directrices de la CNIL.

10 — CONCLUSION ET PERSPECTIVES D'ÉVOLUTION

10.1 Conclusion Générale & Bilan du Projet

La réalisation de Cicados m'a permis de mettre en œuvre un cycle complet de développement web : analyse du besoin, conception, modélisation BDD, développement frontend/backend, authentification, tests et préparation du déploiement. Le projet répond au besoin central de l'établissement en réunissant réservations, catalogue, tournois, support et administration. Il m'a surtout permis de consolider ma capacité à relier les choix techniques aux besoins métier.

10.2 Synthèse du Projet et Bilan Technique

La V1 couvre les fonctionnalités principales : réservation avec détection des chevauchements et allocation de table, agenda des tournois, catalogue de jeux, espace membre, support, administration et stocks. Compétences mobilisées : React, Express/Node.js, MySQL, API REST, JWT/RBAC, Docker, Git/GitHub, CI/CD et tests E2E Puppeteer. Les technologies ont été choisies en fonction des besoins concrets du projet.

10.3 Perspectives d'Évolution (Feuille de Route V2)

Bien que l'application Cicados soit entièrement fonctionnelle, plusieurs axes d'amélioration ont été identifiés pour de futures itérations :

- **Païement en ligne sécurisé (Stripe)** : Intégration du SDK Stripe Checkout pour le règlement par carte bancaire.
- **Notifications Temps Réel WebSockets (Socket.io)** : Serveur WebSockets pour mise à jour instantanée des jauges et messages.
- **Export PDF du Récépissé de Réservation** : Génération dynamique d'un ticket récapitulatif PDF avec QR Code.

12 — ANNEXES DU DOSSIER PROFESSIONNEL

11.1 Annexe 1 : Routes Backend (Express API REST) — Tableau des Endpoints API REST

1. Authentification & Profil Utilisateur

Méthode	Endpoint / Route API	Accès	Description & Rôle
POST	/api/auth/register	Public	Création d'un nouveau compte utilisateur (hachage du mot de passe avec <code>bcrypt</code>).
POST	/api/auth/login	Public	Authentification de l'utilisateur et génération du token JWT.
GET	/api/auth/me	Membre	Récupération des informations de session de l'utilisateur connecté.
PUT	/api/auth/profile	Membre	Mise à jour des coordonnées personnelles du profil membre.
POST	/api/auth/upload-avatar	Membre	Téléversement et enregistrement de la photo de profil (Multer).

2. Réservations & Disponibilité des Tables

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/reservations	Public	Consultation du tableau des créneaux et de l'état d'occupation

			des 4 tables pour une date.
GET	/api/reservations/game-availability	Public	Vérification de la disponibilité spécifique d'un jeu sur un créneau horaire.
GET	/api/reservations/user	Membre	Récupération de l'historique complet des réservations du membre connecté.
POST	/api/reservations	Membre	Création d'une réservation de table avec contrôle SQL anti-surbooking.
PUT	/api/reservations/:id	Membre	Modification des détails (heure, jeu, nb de joueurs) d'une réservation active.
DELETE	/api/reservations/:id	Membre	Annulation d'une réservation par le membre et libération immédiate de la table.

3. Tournois TCG & Animations / Événements

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/tournaments	Public	Liste des tournois TCG (Magic, Pokémon,

			Lorcana, Yu-Gi-Oh!) et jauges de places.
GET	/api/events	Public	Liste des animations et initiations gratuites organisées dans l'établissement.
GET	/api/tournaments/my-registrations	Membre	Liste des tournois auxquels le membre connecté est inscrit.
GET	/api/events/my-registrations	Membre	Liste des événements auxquels le membre connecté est inscrit.
POST	/api/tournaments/:id/register	Membre	Inscription du membre connecté à un tournoi TCG.
DELETE	/api/tournaments/:id/register	Membre	Désinscription du membre connecté d'un tournoi TCG.
POST	/api/events/:id/register	Membre	Inscription du membre connecté à une animation gratuite.
DELETE	/api/events/:id/register	Membre	Désinscription du membre connecté d'une animation gratuite.

4. Messagerie & Support Client (Multi-Tours)

Méthode	Endpoint / Route API	Accès	Description & Rôle
---------	----------------------	-------	--------------------

POST	/api/messages	Membre	Envoi d'un nouveau message au support client via la modale de contact.
GET	/api/messages/my-messages	Membre	Consultation du fil de discussion et des réponses reçues sur le profil membre.
GET	/api/messages/unread-count	Membre	Comptage du nombre de réponses non lues par le membre connecté.
PUT	/api/messages/:id/user-read	Membre	Marquer une réponse administrateur comme lue par l'utilisateur.
POST	/api/messages/:id/user-reply	Membre	Envoyer une relance ou réponse dans le fil de discussion support.
GET	/api/messages/admin	Admin	Récupération de l'ensemble des messages et questions clients (Console Admin).
PUT	/api/messages/admin/:id/read	Admin	Marquer un message client comme lu ou en cours de traitement par l'admin.
POST	/api/messages/admin/:id/reply	Admin	Rédiger et envoyer une réponse officielle à la demande d'un membre.
DELETE	/api/messages/admin/:id	Admin	Supprimer un message ou traiter un fil archivé.

5. Ludothèque & API Proxy BoardGameGeek (BGG)

Méthode	Endpoint / Route API	Accès	Description & Rôle
---------	----------------------	-------	--------------------

GET	/api/boardgames	Public	Consultation du catalogue de 100+ jeux (avec mécanisme de secours JSON sur panne SQL).
GET	/api/bgg/search	Public/Admin	Proxy XML2 vers JSON pour rechercher des jeux sur l'API externe BoardGameGeek.
GET	/api/bgg/thing	Public/Admin	Récupération des détails techniques d'un jeu spécifique via BoardGameGeek.

6. Dashboard Administration & Gestion Centralisée

Méthode	Endpoint / Route API	Accès	Description & Rôle
GET	/api/admin/reservations	Admin	Affichage de toutes les réservations clients du site.
PATCH	/api/admin/reservations/:id	Admin	Mise à jour manuelle du statut d'une réservation (CONFIRMED, CANCELLED).
DELETE	/api/admin/reservations/:id	Admin	Annulation et suppression forcée d'une réservation par l'administrateur.
GET	/api/admin/users	Admin	Liste de tous les comptes utilisateurs inscrits.
PATCH	/api/admin/users/:id	Admin	Modification du rôle d'un utilisateur (USER ↔ ADMIN).
DELETE	/api/admin/users/:id	Admin	Modération et suppression définitive d'un compte membre.

POST	/api/admin/tournaments	Admin	Création d'un nouveau tournoi TCG avec Live Preview.
PUT	/api/admin/tournaments/:id	Admin	Modification des paramètres d'un tournoi existant.
DELETE	/api/admin/tournaments/:id	Admin	Suppression d'un tournoi et annulation en cascade des inscriptions.
POST	/api/admin/events	Admin	Création d'une nouvelle animation ou soirée à thème.
PUT	/api/admin/events/:id	Admin	Modification des informations d'un événement.
DELETE	/api/admin/events/:id	Admin	Suppression d'un événement.
POST	/api/admin/boardgames	Admin	Ajout manuel d'un jeu de société dans le catalogue.
PUT	/api/admin/boardgames/:id	Admin	Édition des détails (joueurs, durée, catégorie) d'un jeu de société.
PATCH	/api/admin/boardgames/:id/stock	Admin	Ajustement rapide du stock disponible en boutique via les boutons + / -.
DELETE	/api/admin/boardgames/:id	Admin	Retrait définitif d'un jeu du catalogue.
POST	/api/admin/boardgames/import-hot	Admin	Importation automatisée du Top 100 des jeux les plus populaires de BoardGameGeek.
POST	/api/admin/upload-image	Admin	Téléversement d'affiches d'événements ou visuels de jeux (Multer).

11.2 Annexe 2 : Modèle Logique de Données, Modèle Conceptuel de Données et Modèle Physique de Données complet

1.1 Entités et Attributs Principaux :

Entité	Attributs Principaux	Rôle / Description
USER	id, email, password, firstname, lastname, pseudo, avatar_url, role	Utilisateurs inscrits (Visiteurs, Membres, Administrateurs).
ROOM	id, name, capacity, description	Tables physiques / espaces de jeu de l'établissement (4 tables).
RESERVATION	id, start_time, end_time, game_type, specific_game, players_count, status	Réservations de tables effectuées par les membres.
TOURNAMENT	id, name, game, date, capacity, price, description	Tournois TCG compétitifs organisés par le bar.
TOURNAMENT_REGISTRATION	id, registered_at	Inscriptions des membres aux tournois TCG.

BOARD_GAME	id, name, min_players, max_players, play_time, category, stock	Ludothèque du café et jeux en vente en boutique.
EVENT	id, name, type, game, date, capacity, price, description	Animations gratuites (avant-premières, drafts, initiations).
EVENT_REGISTRATION	id, registered_at	Inscriptions des membres aux animations et événements.
MESSAGE	id, subject, content, is_read, admin_reply, replied_at, user_read	Échanges support client multi-tours entre membres et admin.

1.2 Associations et Cardinalités :

Association	Entité Source	Cardinalité	Entité Cible	Description de la Relation
Effectuer	USER	(0,N) — (1,1)	RESERVATION	Un utilisateur peut faire 0 à N réservations. Une réservation appartient à 1 seul utilisateur.

Occuper	ROOM	(0,N) — (1,1)	RESERVATION	Une table physique peut être réservée 0 à N fois. Une réservation concerne 1 seule table.
S'inscrire (Tournoi)	USER	(0,N) — (1,1)	TOURNAMENT _REGISTRATION	Un membre peut s'inscrire à 0 ou plusieurs tournois.
Remplir (Tournoi)	TOURNAMENT	(1,N) — (1,1)	TOURNAMENT _REGISTRATION	Un tournoi possède 1 à N inscriptions de joueurs.
S'inscrire (Événement)	USER	(0,N) — (1,1)	EVENT_REGISTRATION	Un membre peut participer à 0 ou plusieurs animations.
Remplir (Événement)	EVENT	(1,N) — (1,1)	EVENT_REGISTRATION	Un événement possède 1 à N participants inscrits.
Envoyer	USER	(0,N) — (1,1)	MESSAGE	Un membre peut poser 0 à N questions au support client.

2. Modèle Logique de Données (MLD)

users (**id**, email, password, firstname, lastname, pseudo, avatar_url, role, created_at)
 rooms (**id**, name, capacity, description, created_at, updated_at)
 reservations (**id**, #user_id, #room_id, start_time, end_time, game_type, specific_game, players_count, status, created_at, updated_at)
 tournaments (**id**, name, game, date, capacity, price, description, created_at)
 tournament_registrations (**id**, #tournament_id, #user_id, registered_at)
 board_games (**id**, name, min_players, max_players, play_time, category, description, image_url, rules_url, stock, created_at)
 events (**id**, name, type, game, date, capacity, price, description, created_at)
 event_registrations (**id**, #event_id, #user_id, registered_at)
 messages (**id**, #user_id, subject, content, is_read, admin_reply, replied_at, user_read, created_at)

3. Modèle Physique de Données (MPD — Dictionnaire de Données MySQL)

3.1 Table `users` (Utilisateurs) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant unique de l'utilisateur.
email	VARCHAR(255)	NON	UNIQUE / INDEX	Adresse e-mail unique de connexion.
password	VARCHAR(255)	NON	-	Mot de passe haché (algorithme bcrypt).
firstname	VARCHAR(100)	NON	-	Prénom du membre.

lastname	VARCHAR(100)	NON	-	Nom de famille du membre.
pseudo	VARCHAR(100)	OUI	-	Pseudonyme public du joueur.
avatar_url	VARCHAR(500)	OUI	-	URL de l'image de profil (Multer upload).
role	ENUM('USER', 'ADMIN')	NON	DEFAULT 'USER'	Rôle applicatif (accès d'administration si ADMIN).
created_at	TIMESTAMP	NON	CURRENT_TIMESTAMP	Date d'inscription du membre.

3.2 Table `reservations` (Réservations de Tables) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant unique de la réservation.

user_id	INT UNSIGNED	NON	FOREIGN KEY	Référence au membre (users.id - CASCADE).
room_id	INT UNSIGNED	NON	FOREIGN KEY	Référence à la table physique (rooms.id - CASCADE).
start_time	DATETIME	NON	INDEX	Date et heure de début de session.
end_time	DATETIME	NON	INDEX	Date et heure de fin de session calculée.
game_type	VARCHAR(50)	OUI	-	Type de jeu (TCG, Jeu de société, BYOG).
specific_game	VARCHAR(255)	OUI	-	Nom exact du jeu sélectionné.
players_count	INT UNSIGNED	NON	DEFAULT 2	Nombre de joueurs réservés pour la table.

status	ENUM('PENDING', 'CONFIRMED', 'CANCELLED'))	NON	DEFAULT 'PENDING'	Statut opérationnel de la réservation.
--------	--	-----	----------------------	---

3.3 Table `messages` (Support Client Multi-Tours) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant du message support.
user_id	INT UNSIGNED	NON	FOREIGN KEY	Référence à l'expéditeur membre (<code>users.id</code>).
subject	VARCHAR(255)	NON	-	Objet du message ou sujet de la question.
content	TEXT	NON	-	Contenu rédigé par l'utilisateur.

is_read	TINYINT(1)	NON	DEFAULT 0	Statut de lecture administrateur (0: non lu, 1: lu).
admin_reply	TEXT	OUI	-	Réponse rédigée par l'équipe d'administration.
replied_at	DATETIME	OUI	-	Date et heure d'envoi de la réponse admin.
user_read	TINYINT(1)	NON	DEFAULT 1	Statut de lecture de la réponse par le membre.
created_at	TIMESTAMP	NON	CURRENT_TIMESTAMP	Horodatage d'envoi du message initial.

3.4 Table `board\_games` (Ludothèque & Stocks Boutique) :

Champ / Colonne	Type SQL	Null	Clé / Index	Description & Rôle
id	INT UNSIGNED AUTO_INCREMENT	NON	PRIMARY KEY	Identifiant unique du jeu.

name	VARCHAR(255)	NON	UNIQUE	Nom officiel du jeu de société.
min_players	INT UNSIGNED	NON	-	Nombre minimal de joueurs.
max_players	INT UNSIGNED	NON	-	Nombre maximal de joueurs.
play_time	INT UNSIGNED	NON	-	Durée estimée d'une partie (minutes).
category	VARCHAR(100)	NON	-	Catégorie (Stratégie, Ambiance, Famille).
stock	INT UNSIGNED	NON	DEFAULT 1	Quantité en stock magasin (boutons + / -).

3.4 Dictionnaire de données de la base de données

Table : messages (Support Client Multi-Tours) :

Gère les échanges de messagerie et de support entre les membres et l'équipe d'administration.

Champ / Colonne	Type SQL	Spécificités / Contraintes	Rôle & Description Fonctionnelle
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique du message.
user_id	INT UNSIGNED	FOREIGN KEY, ON DELETE CASCADE	Identifiant du membre auteur (users.id).
subject	VARCHAR(255)	NOT NULL	Sujet ou titre de la demande de contact.
content	TEXT	NOT NULL	Contenu du message et historique des échanges.
is_read	TINYINT(1)	NOT NULL, DEFAULT 0	0 = Message non lu par l'admin (badge NOUVEAU), 1 = Lu.
admin_reply	TEXT	NULL	Dernier texte de réponse rédigé par l'administrateur.
replied_at	DATETIME	NULL	Date et heure de la dernière réponse de l'admin.

user_read	TINYINT(1)	NOT NULL, DEFAULT 1	0 = Nouvelle réponse admin non lue par le membre, 1 = Lue.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date d'envoi du message initial.

Évolutions des Tables Existantes :

Au cours du développement du projet, la structure de la base de données a naturellement évolué pour s'adapter aux nouveaux besoins fonctionnels et affiner la modélisation des données. Plusieurs tables ont été remaniées et certaines colonnes ont été ajoutées, modifiées ou supprimées afin d'optimiser les requêtes, d'assurer l'intégrité des relations et d'améliorer les performances globales de l'application.

Table Concernée	Colonne Ajoutée	Type SQL	Description & Utilité Fonctionnelle
board_games	stock	INT UNSIGNED, DEFAULT 1	Indique le nombre d'exemplaires du jeu disponibles en magasin, ajustable en 1 clic (+ / -) sur le Dashboard Admin.

users	pseudo	VARCHAR(100) , NULL	Pseudonyme public du membre affiché sur les réservations et classements.
users	avatar_url	VARCHAR(500) , NULL	Lien vers l'image de profil téléversée par le membre via le middleware Multer.

11.3 Annexe 3 : Script SQL Complet de Création et Structuration de la Base de Données ('schema.sql')

Ce script SQL contient l'intégralité du DDL (Data Definition Language) de la base de données relationnelle **Cicados** sous MySQL 8.0, incluant la création des 9 tables relationnelles, les contraintes de clés étrangères avec suppression en cascade (ON DELETE CASCADE), les clés d'unicité et les index composites de performance temporelle :

```
1. CREATE DATABASE IF NOT EXISTS cicados

2. CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3.
4. USE cicados;
5.
6. -- 1. Table des utilisateurs (users)
7. CREATE TABLE IF NOT EXISTS users (
8.     id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
9.     email VARCHAR(255) NOT NULL UNIQUE,
10.    password VARCHAR(255) NOT NULL,
11.    firstname VARCHAR(100) NOT NULL,
12.    lastname VARCHAR(100) NOT NULL,
13.    pseudo VARCHAR(100) DEFAULT NULL,
14.    role ENUM('USER', 'ADMIN') NOT NULL DEFAULT 'USER',
15.    avatar_url VARCHAR(500) DEFAULT NULL,
16.    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
17.    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
18.
19.    INDEX idx_email (email)
20.) ENGINE=InnoDB;
21.
22.-- 2. Table des salles (rooms / tables physiques de jeu)
23.CREATE TABLE IF NOT EXISTS rooms (
24.    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
25.    name VARCHAR(100) NOT NULL,
26.    capacity INT UNSIGNED NOT NULL,
27.    description TEXT,
28.    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
29.    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
30.) ENGINE=InnoDB;
31.
32.-- 3. Table des réservations (reservations)
33.CREATE TABLE IF NOT EXISTS reservations (
34.    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
```

```

35.     user_id INT UNSIGNED NOT NULL,
36.     room_id INT UNSIGNED NOT NULL,
37.     start_time DATETIME NOT NULL,
38.     end_time DATETIME NOT NULL,
39.     game_type VARCHAR(50) DEFAULT 'OTHER',
40.     status ENUM('PENDING', 'CONFIRMED', 'CANCELLED') DEFAULT
    'PENDING',
41.     specific_game VARCHAR(255) DEFAULT NULL,
42.     players_count INT UNSIGNED DEFAULT 2,
43.     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
44.     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
45.
46.     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
47.     FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
48.     INDEX idx_reservation_times (start_time, end_time)
49.) ENGINE=InnoDB;
50.
51.-- 4. Table des tournois (tournaments)
52.CREATE TABLE IF NOT EXISTS tournaments (
53.     id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
54.     name VARCHAR(255) NOT NULL,
55.     game VARCHAR(100) NOT NULL,
56.     date DATETIME NOT NULL,
57.     capacity INT UNSIGNED NOT NULL,
58.     price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
59.     description TEXT,
60.     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
61.) ENGINE=InnoDB;
62.
63.-- 5. Table des inscriptions aux tournois (tournament_registrations)
64.CREATE TABLE IF NOT EXISTS tournament_registrations (
65.     id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
66.     tournament_id INT UNSIGNED NOT NULL,
67.     user_id INT UNSIGNED NOT NULL,
68.     registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
69.     FOREIGN KEY (tournament_id) REFERENCES tournaments(id) ON DELETE
    CASCADE,
70.     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
71.     UNIQUE KEY idx_tourney_user (tournament_id, user_id)
72.) ENGINE=InnoDB;
73.
74.-- 6. Table des jeux de société (board_games)
75.CREATE TABLE IF NOT EXISTS board_games (
76.     id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,

```

```

77.     name VARCHAR(255) NOT NULL UNIQUE,
78.     min_players INT UNSIGNED NOT NULL,
79.     max_players INT UNSIGNED NOT NULL,
80.     play_time INT UNSIGNED NOT NULL,
81.     category VARCHAR(100) NOT NULL,
82.     description TEXT,
83.     image_url VARCHAR(500) DEFAULT NULL,
84.     rules_url VARCHAR(500) DEFAULT NULL,
85.     stock INT UNSIGNED NOT NULL DEFAULT 1,
86.     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
87.) ENGINE=InnoDB;
88.
89.-- 7. Table des événements (events)
90.CREATE TABLE IF NOT EXISTS events (
91.    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
92.    name VARCHAR(255) NOT NULL,
93.    type ENUM('avant_premiere', 'draft', 'initiation') NOT NULL,
94.    game VARCHAR(100) NOT NULL,
95.    date DATETIME NOT NULL,
96.    capacity INT UNSIGNED NOT NULL,
97.    price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
98.    description TEXT,
99.    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
100.) ENGINE=InnoDB;
101.
102.-- 8. Table des inscriptions aux événements (event_registrations)
103.CREATE TABLE IF NOT EXISTS event_registrations (
104.    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
105.    event_id INT UNSIGNED NOT NULL,
106.    user_id INT UNSIGNED NOT NULL,
107.    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
108.    FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
109.    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
110.    UNIQUE KEY idx_event_user (event_id, user_id)
111.) ENGINE=InnoDB;
112.
113.-- 9. Table du support messagerie (messages)
114.CREATE TABLE IF NOT EXISTS messages (
115.    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
116.    user_id INT UNSIGNED NOT NULL,
117.    subject VARCHAR(255) NOT NULL,
118.    content TEXT NOT NULL,
119.    is_read TINYINT(1) NOT NULL DEFAULT 0,
120.    admin_reply TEXT DEFAULT NULL,

```

```

121.     replied_at DATETIME DEFAULT NULL,
122.     user_read TINYINT(1) NOT NULL DEFAULT 1,
123.     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
124.     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

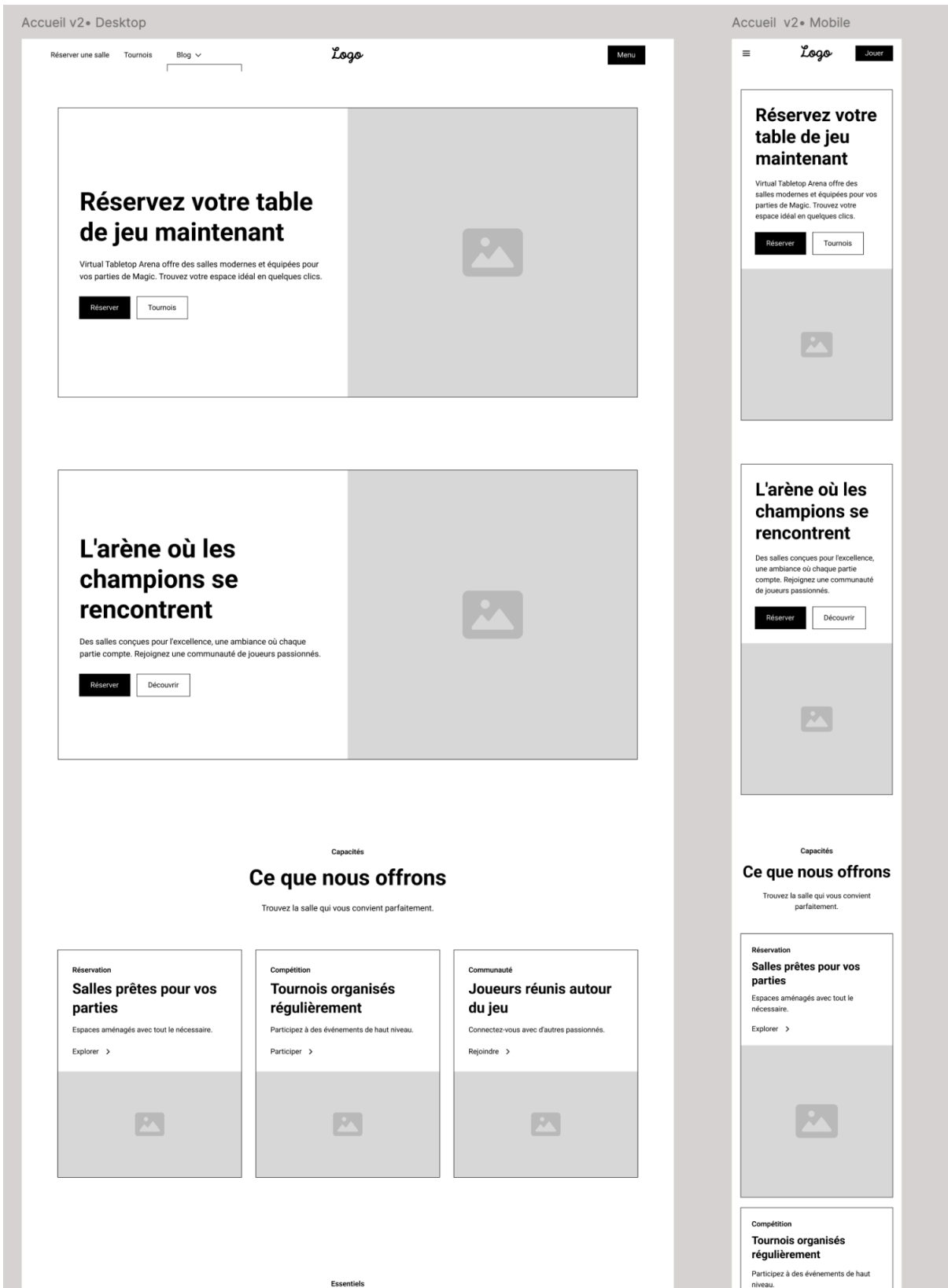
```

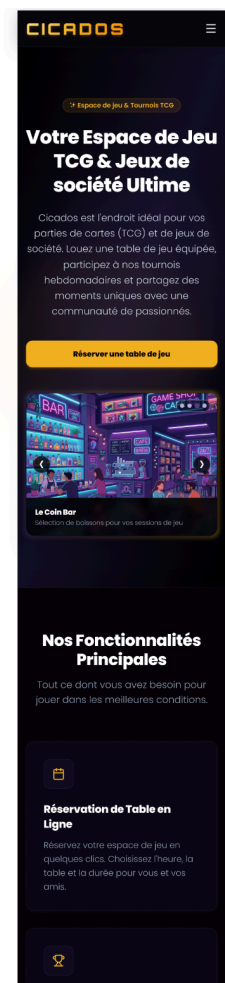
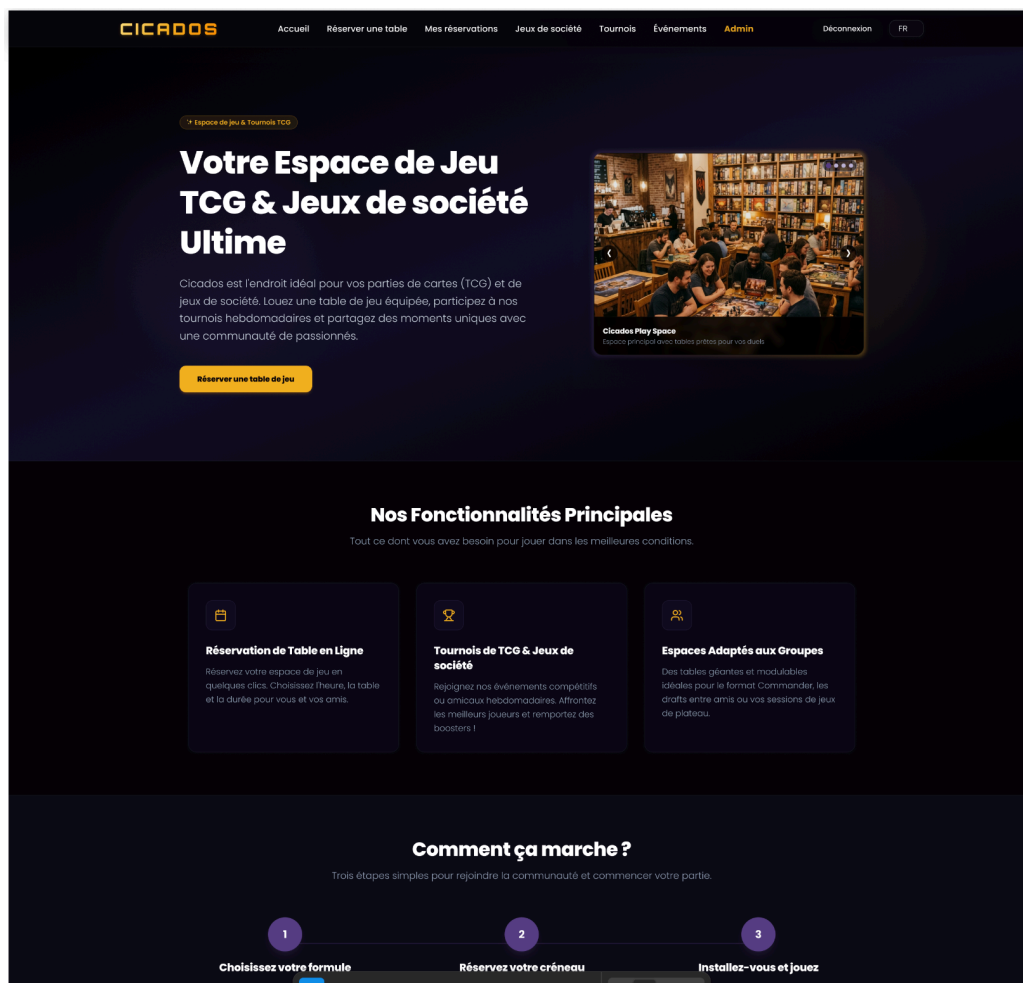
) ENGINE=InnoDB;

11.4 Annexe 4 : Planches Figma — Wireframes Low-Fidelity & Maquettes High-Fidelity UI/UX

Cette planche de conception UI/UX présente le processus de maquettage réalisé sur Figma, depuis le découpage de la grille d'alignement responsive jusqu'aux maquettes Haute Fidélité :







Composants & Spécifications du Design System Figma :

- **Grille & Layout** : Grille 12 colonnes adaptative pour les résolutions Desktop ($\geq 1024\text{px}$) et 4 colonnes pour les terminaux mobiles ($\leq 640\text{px}$).
- **Palette de Couleurs (Tokens)** :
 - *Fond Principal* : Violet Sombre #201848 / Noir d'Encre #0f172a.
 - *Accents Secondaires* : Jaune Ambré #FFBF00
 - *Cartes et Modales* : Fond dépoli semi-transparent avec bordures dorées fines (1px solid #d97706).
- **Composants Clés Modélisés** : Modale de réservation avec calendrier sélecteur interactif, console d'administration 6 onglets avec ajustement instantané des stocks, et espace support réactif.

11.5 Annexe 5 : Script de Tests Automatisés End-to-End avec Puppeteer Chrome Headless

Cet extrait de script Node.js utilise le framework **Puppeteer** pour valider automatiquement le parcours utilisateur sans intervention humaine (connexion, détection de conflit de table et génération de captures d'écran de preuve) :

```
125. // tests/e2e/reservation.test.js - Validation Automatisée E2E
    Puppeteer

126. const puppeteer = require('puppeteer');
127.
128. (async () => {
129.     console.log('🚀 Lancement de la suite de tests E2E
        Puppeteer...');
130.
131.     const browser = await puppeteer.launch({ headless: true });
132.     const page = await browser.newPage();
133.     await page.setViewport({ width: 1280, height: 800 });
134.
135.     // Étape 1 : Navigation vers la page de connexion
136.     await page.goto('http://localhost:5173/login');
137.     await page.type('#email', process.env.TEST_USER_EMAIL);
138.     await page.type('#password', process.env.TEST_USER_PASSWORD);
139.     await page.click('button[type="submit"]');
140.
141.     await page.waitForNavigation();
142.     console.log('✅ Authentification JWT réussie avec succès.');
```

```
143.
144.     // Étape 2 : Test du formulaire de réservation de table
145.     await page.goto('http://localhost:5173/reservations');
146.     await page.type('#date', '2026-09-15');
147.     await page.select('#timeSlot', '14:00');
148.     await page.click('#submitBooking');
149.
150.     // Étape 3 : Capture d'écran de confirmation et assertion
151.     await page.waitForSelector('.toast-success');
152.     await page.screenshot({ path:
        'tests/screenshots/reservation_confirmed.png' });
153.
154.     console.log('🎉 Réservation validée et capture d\'écran
        enregistrée !');
```

```
155.     await browser.close();

    }) ();
```

11.6 Annexe 6 : Fichier de Configuration des Variables d'Environnement (.env.example)

Structure des variables d'environnement requises pour le déploiement sécurisé du backend Node.js / Express :

```
156. # Configuration du Serveur Express API REST

157. PORT=5000
158. NODE_ENV=production
159. CORS_ORIGIN=CHANGE_ME
160.
161. # Connexion Base de Données MySQL (mysql2 pool)
162. DB_HOST=DB_HOST
163. DB_PORT=DB_PORT
164. DB_USER=DB_USER
165. DB_PASSWORD=DB_PASSWORD
166. DB_NAME=DB_NAME
167.
168. # Clés Sécurité & Authentification JWT
169. JWT_SECRET=CHANGE_ME
170. JWT_EXPIRES_IN=CHANGE_ME
171.
172. # Configuration Clé API BoardGameGeek (BGG Proxy)
173. BGG_API_URL=CHANGE_ME
174.
175. # Répertoire de Stockage Médias (Multer
    Upload)UPLOAD_PATH=./CHANGE_ME
```